Less is More: Small Language Models as the Substrate of Agentic AI—A Survey of Architectures, Compression, Deployment, and Reported Performance

Mohammed Gamal Ragab [1,2*], **Hitham Alhussian** [1,2*], **Said Jadid Abdulkadir** [1,2], **and Ayed Alwadain** [4]

[1]Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[2]Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[4]Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia
[*]Corresponding authors: mohammed.ragab@utp.edu.my (M.G.R.); seddig.alhussian@utp.edu.my (H.A.)

Abstract

Large Language Models (LLMs) with hundreds of billions of parameters have advanced natural language processing dramatically, but their computational cost, memory demand, and latency make them a poor fit for the fastest-growing consumer of language intelligence: the AI agent, which issues many narrow, repetitive, well-specified model calls rather than a few open-ended ones. Small Language Models (SLMs)—here, decoder-only or hybrid transformers with roughly twenty billion parameters or fewer, counting active parameters for mixture-of-experts models—have closed much of the capability gap through knowledge distillation, synthetic-data curation, and post-training quantization, and are now the natural default for agentic workloads. This paper is a combined survey and position statement. We catalog 123 publicly documented SLM variants released between 2022 and 2026, including the 2025–2026 wave of hybrid state-space models (Falcon-H1, IBM Granite 4.0, NVIDIA Nemotron Nano 2), unified reasoning models (Qwen3, Phi-4-mini-reasoning, SmolLM3), and mobile-first multimodal families (Gemma 3n, Gemma 4, Apple Foundation Models). We organize the landscape along a four-dimensional taxonomy—*architecture*, *training strategy*, *modality*, and *application domain*—and review the compression-technique literature specific to the sub-10B regime, in which quantization consistently outperforms pruning. We then consolidate publicly *reported* performance for a 17-model cohort across 15 benchmark tasks, three evaluation paradigms, six hardware tiers, and three quantization precisions, under a documented grey-literature protocol; every figure is a reported value traced to a dated source rather than an original benchmark run. Building on this evidence, we argue that SLMs are the substrate of agentic AI: they meet the capability threshold for the sub-tasks agents actually delegate, they cost 10–30× less to serve, they are cheaper to specialize, and their behavior is easier to constrain to tool and code interfaces. We survey heterogeneous SLM–LLM architectures, learned routing, and the LLM-to-SLM conversion pipeline, and close with the open problems—reasoning under parameter constraints, hallucination, hardware co-design, fairness under compression, and empirical verification of the consolidated evidence base—that stand between the current state of the art and that future.

Highlights

- Surveys 123 documented small language model variants (2022–2026), including the 2025–2026 hybrid state-space, unified-reasoning, and mobile-multimodal waves, under a taxonomy of architecture, training strategy, modality, and application domain.
- Consolidates publicly reported performance for 17 SLMs across 15 tasks, six hardware tiers, and FP16/INT8/INT4 precision, with every figure traced to a dated source via a documented protocol.
- Reviews compression for the sub-10B regime and the finding that quantization consistently outperforms pruning below 4B parameters.
- Argues that SLMs are the substrate of agentic AI on grounds of capability sufficiency, 10–30× lower serving cost, cheaper specialization, and tighter behavioral alignment to tool interfaces.
- Surveys heterogeneous SLM–LLM routing and the LLM-to-SLM conversion pipeline, and sets an empirical-verification agenda for the consolidated claims.

Keywords: Small language models • Agentic AI • Model compression • Edge AI • On-device inference • LLM–SLM collaboration

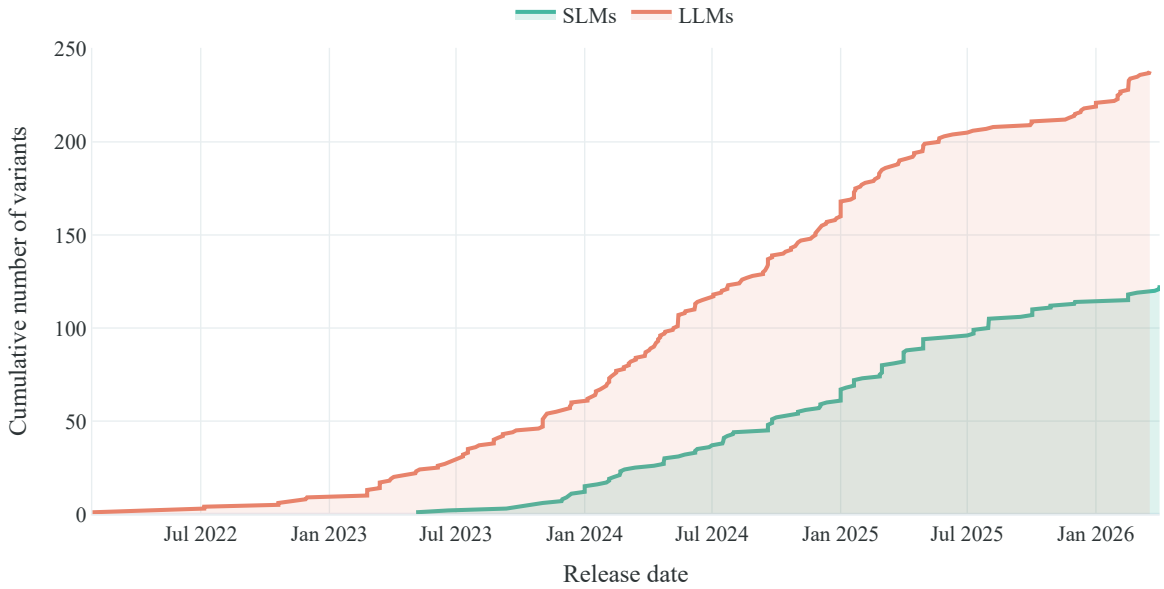


Figure 1. Cumulative count of documented small (SLM) and large (LLM) language-model variants by release date, from the model inventory of Appendix B (snapshot September 2026). SLM releases accelerate sharply from mid-2024.

1 Introduction

The landscape of natural language processing (NLP) has been reshaped by large language models (LLMs). Systems such as GPT-4 [1], Claude [2], Gemini [3], and the LLaMA series [4] demonstrate broad competence across reasoning, code generation, and multilingual tasks. That competence, however, comes at extraordinary cost. GPT-3's 175 billion parameters [5] inaugurated an era in which scale was treated as synonymous with capability, and scaling laws [6, 7] appeared to promise predictable returns on ever-larger models. Frontier LLMs now require multi-thousand-GPU clusters to train, tens of gigabytes of accelerator memory to serve, and cloud infrastructure that introduces latency, recurring cost, and privacy exposure. Training GPT-3 is estimated to have consumed 1,287 MWh and emitted roughly 502 metric tons of CO_2 equivalent [8]; a single frontier-model query draws on the order of ten times the energy of a conventional web search [9].

1.1 The Rise of Small Language Models

Running against this trajectory is a counter-trend: small language models (SLMs) that achieve disproportionately strong performance for their size. We use *SLM* to mean a decoder-only or hybrid transformer with roughly twenty billion parameters or fewer—counting *active* parameters for mixture-of-experts models, since deployment cost tracks those—small enough to be plausibly served on a single consumer GPU, a laptop, or a modern phone. The empirical cohort of Section 5 is concentrated below 10B, with Phi-4 (14B) retained as an upper-bound reference. Where earlier compact models such as DistilBERT [10] and TinyBERT [11] were task-specific encoders, the current generation are general-purpose, instruction-following systems. The Chinchilla scaling analysis [7] showed that most large models were badly undertrained and that a smaller model given proportionally more data could match them; the Phi family then demonstrated that "textbook-quality" synthetic data lets a 3.8B model rival models an order of magnitude larger [12], Gemma applied distillation from larger teachers [13], and Qwen2.5 pushed sub-7B training to 18 trillion tokens [14]. MIT Technology Review named SLMs one of its 10 Breakthrough Technologies of 2025 [15]. By 2026 the strongest 3–4B models post benchmark scores that were state-of-the-art for 13B-class models in 2023, and the 2025–2026 release wave adds hybrid state-space architectures [16], controllable-reasoning models [17], and mobile-first multimodal families [18, 19]. Figure 1 shows the resulting acceleration: the cumulative count of documented open SLM variants has grown from a handful in 2023 to more than one hundred by early 2026.

1.2 The Agentic Turn

At the same time, the way language models are consumed has shifted. The fastest-growing workload is not the chat session but the *AI agent*: a control loop in which a model is called repeatedly to choose a tool, fill its arguments, extract a field, classify an intent, or check a result [20]. These calls are narrow, repetitive, and format-constrained,

and they exercise only a thin slice of a frontier model's competence. Belcak, Heinrich, Diao, et al. [20] argue—and this paper concurs and elaborates—that for such workloads SLMs are not a compromise but the appropriate default: sufficiently capable for the delegated sub-task, 10–30× cheaper to serve, cheaper to specialize, and easier to constrain to a code interface, with a strong generalist model reserved for the minority of calls that genuinely need planning or broad context. Surveys of LLM–SLM collaboration [21, 22] and production systems such as Apple's on-device/server split [19] indicate that this heterogeneous design is already the direction of travel.

1.3 Scope and Contributions

Existing SLM surveys [23–25] provide taxonomies and measurements but predate the 2025–2026 model wave and treat agentic deployment only in passing; agentic position pieces [20] make the argument but do not ground it in a consolidated capability-and-deployment review. This paper joins the two. It is simultaneously a survey of the SLM landscape and a position statement that SLMs are the substrate of agentic AI. Its contributions are:

Landscape and taxonomy
: A catalog of 123 documented SLM variants (2022–2026), including the recent hybrid, reasoning, and mobile-multimodal families, organized along a four-dimensional taxonomy of architecture, training strategy, modality, and application domain (Section 3 and Appendix B).

Compression review
: A synthesis of distillation, quantization, pruning, and compound pipelines for the sub-10B regime, including the finding that quantization consistently outperforms pruning below 4B parameters [26] (Section 4).

Consolidated reported performance
: A comparison of 17 SLMs across 15 tasks, three evaluation paradigms, six hardware tiers, and three precisions, assembled under a documented grey-literature protocol in which every figure is a *reported* value traced to a dated source rather than an original run (Section 5, Appendix A).

Deployment ecosystem
: A survey of inference frameworks, quantization formats, and on-device platforms from Raspberry Pi to Apple silicon (Section 6).

The agentic argument
: A structured case that SLMs are the default execution substrate for agents, covering capability sufficiency, economics, behavioral alignment, heterogeneous SLM–LLM routing, and the LLM-to-SLM conversion pipeline (Section 7).

Open problems and verification
: An agenda of open research problems and targeted empirical-verification opportunities for the consolidated evidence base (Section 10).

1.4 Paper Organization

Section 2 reviews prior work on the LLM-to-SLM transition, compression, edge inference, and LLM–SLM collaboration. Section 3 presents the four-dimensional taxonomy. Section 4 reviews compression techniques. Section 5 documents the data-collection protocol and consolidates reported performance. Section 6 covers resource-constrained deployment. Section 7 makes the agentic argument. Section 8 surveys domain-specific applications. Section 9 discusses implications, limitations, and ethics. Section 10 sets out open challenges and an empirical-verification agenda. Section 11 concludes.

The ecosystem inventory referenced throughout is generated directly from `merged_llm_database.json` (snapshot September 2026). Appendix B provides the full variant inventory grouped chronologically by release year and family, together with the source-selection protocol.

2 Related Work and Research Gaps

2.1 Evolution from LLMs to SLMs

The modern era of language modeling began with the Transformer architecture [27], which replaced recurrent networks with self-attention mechanisms enabling efficient parallel training. BERT [28] demonstrated the power of pre-training on large corpora followed by task-specific fine-tuning, establishing a paradigm that dominated NLP from 2018 to 2020. The GPT series [1, 5, 29] progressively scaled this approach, with GPT-3 (175B parameters) revealing emergent capabilities—such as in-context learning and chain-of-thought reasoning—that appeared to arise only at sufficient scale. This observation motivated an aggressive scaling race: PaLM (540B) [30], Chinchilla (70B) [7], LLaMA (7B–65B) [31], LLaMA-2 (7B–70B) [32], Falcon (7B–180B) [33], and ultimately GPT-4 [1] and Gemini

Ultra [3] pushed parameter counts into the hundreds of billions.

However, the scaling paradigm encountered critical challenges. Kaplan et al. [6] established power-law relationships between model performance and three factors: parameter count, training data size, and compute budget. Crucially, Hoffmann et al. [7] revised these scaling laws with the Chinchilla study, demonstrating that for a fixed compute budget, models and training data should be scaled in equal proportion. Their 70B-parameter Chinchilla model, trained on 1.4 trillion tokens, outperformed the 280B-parameter Gopher model trained on only 300 billion tokens. This finding implied that most existing LLMs were significantly under-trained and that smaller, better-trained models could achieve equivalent performance at lower cost. Recent work has further validated and refined this principle: the Qwen2.5 series trains its 7B model on 18 trillion tokens [14]—over 2,500× the parameter count in tokens—far exceeding the Chinchilla-optimal ratio and demonstrating that the benefits of extended training persist well beyond theoretical optima.

The counter-trend toward smaller models accelerated in 2023–2024 with the release of several pivotal SLM families. Microsoft's Phi-1 (1.3B) [34] demonstrated that a small model trained on carefully curated "textbook-quality" data could achieve surprisingly strong performance on code generation tasks. Phi-2 (2.7B) [35] extended this to general language understanding, matching or exceeding models with 25× more parameters on certain benchmarks. The Phi-3 family [12] formalized the synthetic-data-first training philosophy, with Phi-3-mini (3.8B) achieving performance comparable to Mixtral 8×7B (46.7B total parameters) on Massive Multitask Language Understanding (MMLU). Google's Gemma [13] and Gemma 2 [36] leveraged distillation from Gemini models, while Meta's LLaMA-3.2 [4] introduced 1B and 3B variants specifically designed for edge deployment. By early 2026, the Phi-4-mini (3.8B) achieves 88.6% on Grade School Math 8K (GSM8K) mathematical reasoning [37], a score that would have placed it among the top models at any parameter count in 2023.

2.2 Knowledge Distillation and Model Compression

Knowledge distillation, first formalized by Hinton et al. [38], transfers knowledge from a large "teacher" model to a smaller "student" by training the student to match the teacher's soft probability distributions (logits) rather than hard labels. This approach preserves the teacher's learned representations of inter-class similarities—information that is lost in one-hot labels—and has proven remarkably effective in the LLM era. Modern distillation strategies for language models encompass several complementary approaches.

Logit-based distillation trains the student to minimize the KL divergence between its output distribution and the teacher's softened distribution, parameterized by a temperature τ. Google's Gemma 2 [36] and Gemma 3 [39] employ this approach extensively: all Gemma 3 model sizes, including the 27B variant, are trained using knowledge distillation from a larger teacher. The distillation loss is combined with a standard next-token prediction loss, with the relative weighting serving as a key hyperparameter. Feature-based distillation [40] additionally aligns intermediate representations between teacher and student, capturing structural knowledge that may not be fully expressed in output logits. Microsoft's Orca 2 [41] demonstrated a variant where a smaller model is taught to use different reasoning strategies depending on task complexity, outperforming models with 5–10× more parameters on reasoning benchmarks.

Synthetic data generation represents an increasingly dominant form of implicit distillation. Rather than direct logit matching, a powerful LLM (e.g., GPT-4, Claude 3.5) generates high-quality training examples that are then used to train smaller models. The Phi series pioneered this approach with the "Textbooks Are All You Need" methodology [34], generating synthetic textbook-style content with carefully controlled complexity and diversity. Phi-4's training data comprises approximately 40% synthetic data generated across 50 distinct dataset types, including multi-agent conversations, structured reasoning chains, and domain-specific problem sets [42]. This synthetic-data-first approach has proven more effective than distillation from raw web corpora, particularly for STEM and reasoning tasks.

Post-training compression techniques enable further size reduction without retraining. Quantization reduces the numerical precision of model weights from 32-bit or 16-bit floating point to lower bit-widths. GPTQ [43] performs layer-wise quantization using approximate second-order (Hessian) information, achieving INT4 precision with minimal perplexity increase. activation-aware weight quantization (AWQ) [44] identifies and protects the small fraction (~1%) of salient weights determined by activation magnitudes, achieving near-lossless INT4 quantization and winning the MLSys 2024 Best Paper award. GPT-generated unified format (GGUF), developed

for the llama.cpp ecosystem [45], provides block-based K-quant schemes (Q4_K_M, Q5_K_M, Q8_0) optimized for CPU inference with fine-grained quality–size trade-offs. SqueezeLLM [46] introduced a sensitivity-based non-uniform quantization scheme with dense-and-sparse decomposition, demonstrating that sensitivity varies dramatically across weight dimensions.

Complementary compression techniques include structured and unstructured pruning, which remove redundant weights or entire attention heads [47]; low-rank adaptation (LoRA) [48] and its 4-bit variant quantized low-rank adaptation (QLoRA) [49], which enable parameter-efficient fine-tuning by learning low-rank update matrices while keeping the base model frozen; and vocabulary pruning, which reduces embedding table size for domain-specific deployments. The combination of distillation during training, quantization during deployment, and LoRA-based adaptation for task specialization forms the modern SLM optimization pipeline.

2.3 Efficient Fine-Tuning Methods

Full fine-tuning updates all model parameters using task-specific data, which for a 7B-parameter model can require tens of gigabytes of accelerator memory once gradients and optimizer states are included. This cost has made parameter-efficient fine-tuning (PEFT) central to the practical adaptation of SLMs, particularly when the target deployment environment is a single GPU or an edge-oriented workstation.

LoRA [48] addresses this constraint by freezing the pretrained backbone and learning low-rank update matrices for selected linear layers. For a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is parameterized as $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters while preserving the ability to specialize a model for downstream tasks. QLoRA [49] extends this idea by quantizing the frozen base model to 4-bit NormalFloat (NF4) precision and training the adapters in higher precision, allowing models that would ordinarily require multi-GPU setups to be fine-tuned on a single high-memory accelerator.

Beyond LoRA-style adapters, the PEFT landscape includes prompt-based and prefix-based adaptation strategies that shift task information into learned soft prompts or attention prefixes rather than updating the backbone itself. These approaches can be attractive when multiple lightweight task specializations must be maintained simultaneously. However, they tend to underperform LoRA-family methods on complex reasoning tasks, where direct adaptation of attention and feed-forward projections provides a stronger inductive bias for task transfer.

The choice between prompting and fine-tuning therefore depends on task specificity, data availability, and deployment constraints. In-context learning offers zero-training adaptation and is well suited to broad tasks when labeled data are scarce, but it consumes context length and raises inference latency. By contrast, PEFT methods incur an upfront training cost yet reduce per-query overhead and typically yield larger gains on narrow or reasoning-heavy tasks. This trade-off motivates the comparative synthesis in Section 5, where reported prompting and QLoRA results are placed side by side.

2.4 Overview of Key SLM Families

2.4.1 Microsoft Phi Series

The Phi series represents Microsoft's systematic exploration of data-quality-driven model development. Phi-1 (1.3B) [34] demonstrated strong code generation performance through training on synthetic "textbook-quality" data. Phi-2 (2.7B) [35] extended this to general language understanding. Phi-3-mini (3.8B) [12] formalized the approach with 3.3 trillion training tokens, achieving 68.8% on MMLU and 82.5% on GSM8K with instruction tuning. Phi-3.5-mini [50] added 128K context length support with improved multilingual capabilities. Phi-4 (14B) [42] introduced a fundamentally enhanced training pipeline incorporating ~40% synthetic data, achieving 84.8% on MMLU and 80.4% on MATH—surpassing its teacher model GPT-4o on several STEM benchmarks. Phi-4-mini (3.8B) [37] updated the architecture with grouped-query attention (GQA) (24 query, 8 key–value (KV) heads), a 200K-token vocabulary, and training on 5 trillion tokens, achieving 88.6% on GSM8K. The Phi-4-reasoning variant [51] applied supervised fine-tuning on o3-mini reasoning demonstrations followed by Group Relative Policy Optimization (GRPO), reaching 82.5% on AIME 2025—competitive with DeepSeek-R1 at 671B parameters.

2.4.2 Google Gemma Series

Google's Gemma family [13] derives from the Gemini model architecture and training infrastructure. Gemma 1 offered 2B and 7B variants trained on 3T and 6T tokens respectively, using Multi-Query Attention (MQA) for the 2B model and standard Multi-Head Attention for the 7B. Gemma 2 [36] introduced significant architectural improvements: local-global interleaved attention layers, logit soft-capping, and knowledge distillation for the 2B

and 9B variants (from a larger teacher model). Gemma 2-9B achieved 71.3% on MMLU (5-shot), establishing it as a strong 7B-class competitor. Gemma 3 [39] further advanced the architecture with a 5:1 local-to-global attention interleaving ratio (reducing the sliding window from 4096 to 1024 tokens), Rotary Position Embeddings, and logit soft-capping at 30.0 for attention and 50.0 for the final layer. Critically, Gemma 3 distills all model sizes from larger teachers and introduces a 1B variant that achieves 48.0% on MATH—nearly matching Gemma 2-9B's 49.4% at 9× fewer parameters.

2.4.3 Alibaba Qwen Series

The Qwen2 [52] and Qwen2.5 [14] families from Alibaba represent the most extensively scaled SLM training effort, with models trained on 18 trillion tokens—substantially exceeding Chinchilla-optimal ratios. Qwen2.5 offers models at 0.5B, 1.5B, 3B, 7B, 14B, 32B, and 72B parameters, all sharing a consistent GQA-based architecture with rotary position embeddings (RoPE) and SwiGLU activations. The 3B variant achieves remarkable results: 65.6% on MMLU, 79.1% on GSM8K, and 42.6% on MATH—scores that establish it as arguably the strongest sub-4B base model available. Qwen2.5's training data includes substantial STEM and code content, explaining its particular strength on mathematical and programming benchmarks.

2.4.4 Other Significant SLM Families

TinyLlama (1.1B) [53] adopted the LLaMA-2 architecture and trained on 3 trillion tokens from SlimPajama and StarCoder data, demonstrating that extended training of very small models yields disproportionate gains. Mistral 7B [54] introduced Grouped-Query Attention and Sliding Window Attention in a 7.3B-parameter model, achieving performance competitive with LLaMA-2 13B while being 1.8× smaller. Meta's Llama 3.2 [4] released dedicated 1B and 3B models trained on 9 trillion tokens with 128K context length, specifically targeting edge deployment. SmolLM2 (135M, 360M, 1.7B) [55] from Hugging Face trained on 11 trillion tokens of carefully curated data. H2O-Danube (1.8B) [56] introduced sliding window attention in a compact model trained on 1 trillion tokens. OLMo 2 (1B, 7B, 13B) [57] from AI2 was released as a fully open model (data, code, weights, and training logs), achieving competitive performance through QK-norm and carefully staged training. StableLM-2 (1.6B) [58] was trained on 4 trillion tokens of multilingual data. The BLOOM family (560M, 1.1B, 1.7B, 3B, 7.1B) [59] remains notable as a multilingual model trained on 46 languages using ALiBi positional encoding, though its 350B-token training budget is modest by current standards.

2.5 Edge AI and On-Device Inference

The edge AI inference stack has matured rapidly, transforming SLM deployment from a research exercise to a production-ready capability. The llama.cpp framework [45] remains the de facto standard for CPU and mixed CPU-GPU inference, supporting the GGUF quantization format with dozens of precision variants (Q2_K through Q8_0). Its ecosystem includes llamafile (single-executable packaging), Ollama (user-friendly serving), and extensive platform support spanning x86, ARM, Apple Silicon, and NVIDIA GPUs. Benchmarking on single-board computers shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models [60].

Meta's ExecuTorch [61] reached version 1.0 GA in October 2025, providing a production-ready framework for on-device AI with a 50 KB base runtime and support for 12+ hardware backends including CoreML, Qualcomm QNN, ARM, MediaTek, and Vulkan. It is deployed in production across Instagram, WhatsApp, and Facebook, serving billions of users. MLC-LLM [62] provides universal deployment through ML compilation (TVM), supporting iOS, Android, web, and embedded platforms. ONNX Runtime [63] enables cross-platform enterprise deployment with optimization passes for diverse hardware targets. WebLLM [64] brings browser-based inference using WebGPU acceleration, retaining up to 80% of native performance with an OpenAI-compatible API. Google's LiteRT (formerly TensorFlow Lite) with QNN acceleration achieves 100× CPU speedup on Qualcomm neural processing units (NPUs) [65], enabling sub-second time-to-first-token for sub-1B models on flagship smartphones.

The fundamental constraint in edge deployment is memory bandwidth rather than compute. Mobile devices offer 50–90 GB/s of memory bandwidth versus 2–3 TB/s for datacenter GPUs [66]. Since autoregressive inference is memory-bandwidth-bound (each token generation requires reading all model weights), quantization from 16-bit to 4-bit provides a near-linear 4× speedup by reducing memory transfer volume. This makes INT4 quantization not merely a storage optimization but a critical performance enabler for on-device deployment.

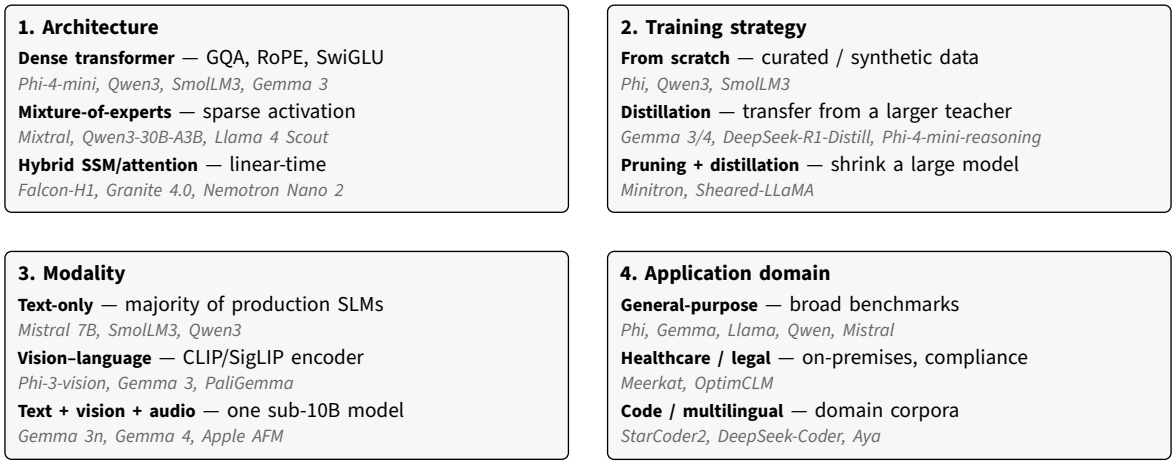


Figure 2. The four-dimensional taxonomy of small language models. The axes are orthogonal: each model occupies one category on each. Representative models are italicised. Section 3 develops each axis; Table 1 gives the tabular form.

2.6 LLM–SLM Collaboration and Agentic Systems

A distinct line of work studies how small and large models cooperate rather than compete. Chen and Varoquaux [21] frame the relationship along two axes—collaboration and competition—and argue that small models remain practically indispensable despite the scaling race. Chen, Zhao, and Han [22] organize the collaboration literature into five interaction patterns: pipeline, routing, auxiliary, distillation, and fusion. Routing is the most directly relevant to agents: RouteLLM [67] learns from preference data to send each query to a weak or strong model, reporting more than 2× cost reduction at matched quality, and Division-of-Thoughts [68] routes decomposed sub-questions between an on-device SLM and a cloud LLM. Surveys of edge–cloud collaborative inference [66] catalog the systems techniques—speculative decoding, cascades, KV-cache offload—that make the hybrid practical. On the agentic side specifically, Belcak, Heinrich, Diao, et al. [20] argue that SLMs should be the default model inside agents, and action-model families such as xLAM [69] show that sub-10B models can top function-calling leaderboards. This paper connects that argument to a consolidated capability-and-deployment review (Sections 5–7).

2.7 Gaps in Existing Literature

Several SLM surveys appeared in 2024–2025. Lu, Li, Cai, et al. [23] analyze 70 models on architecture, capability, and on-device runtime, but as a measurement study rather than a synthesis of the wider reported record. Nguyen, Shen, Aponte, et al. [24] and Wang, Zhang, Zhang, et al. [25] provide broad technique taxonomies and application coverage. All three predate the 2025–2026 wave of hybrid state-space, unified-reasoning, and mobile-multimodal models, and none foreground agentic deployment. Position pieces on SLMs in agents [20] make the forward-looking argument but do not ground it in a consolidated capability review. This paper addresses four gaps: (1) a landscape and taxonomy current to 2026, including hybrid and reasoning families; (2) a single consolidated view of *reported* accuracy *and* deployment behavior across many models, tiers, and precisions, with per-figure provenance; (3) an explicit bridge from that evidence to the agentic case; and (4) a practitioner-facing decision and verification agenda.

3 Taxonomy of Small Language Models

We propose a four-dimensional taxonomy of SLMs organized along orthogonal axes: *architecture*, *training strategy*, *modality*, and *application domain* (Figure 2). A given model may be positioned along all four dimensions simultaneously—Phi-3-vision, for instance, is a dense transformer (architecture), trained from scratch on synthetic data (training strategy), multimodal (modality), and general-purpose (domain).

Figure 3 shows how the size distribution of new SLM releases has evolved: the sub-1B and 1–3B bands dominated early releases, but the 3–7B band has become the centre of gravity as capability-per-parameter has improved. Figure 4 shows the parameter span of the major families.

3.1 Taxonomy by Architecture

3.1.1 Dense Transformer Models

The majority of production SLMs use a dense transformer decoder—every token attends to every other token (within window) and every parameter is activated per forward pass. This category spans from sub-1B models

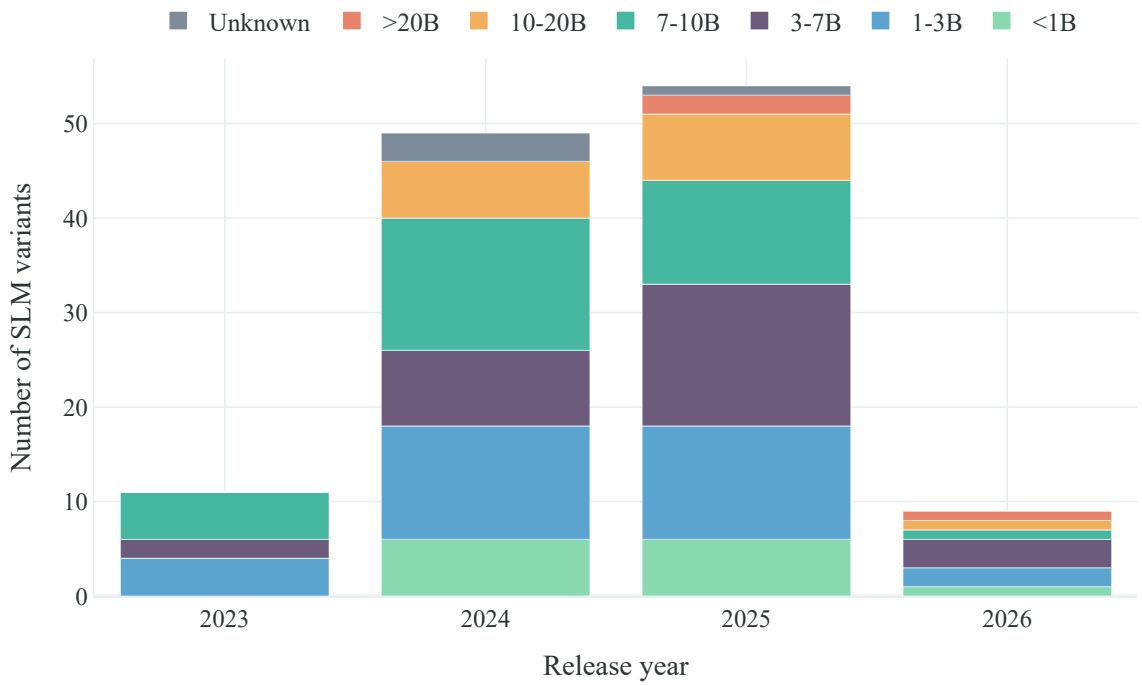


Figure 3. Documented SLM variants by release year and parameter band. The 3–7B band has grown from a minority of releases in 2023 to the largest single band by 2025.

(SmolLM2-135M [70]) to 7–8B flagships (Mistral 7B [54], Llama 3.1 8B [4]). Recent dense families are notable for pushing capability per parameter: SmolLM3-3B [71] uses grouped-query attention with NoPE and is trained on 11.2T tokens with a documented three-stage curriculum, matching or exceeding Llama-3.2-3B and Qwen2.5-3B; Qwen3's dense line [17] spans 0.6B–32B under a single architecture combining GQA, RoPE, and SwiGLU with a unified thinking / non-thinking switch. Key architectural improvements over the vanilla GPT-2 design include:

Grouped-query attention
: Introduced in LLaMA-2 [32] and adopted by Mistral 7B and most post-2023 SLMs, GQA reduces the KV cache memory by a factor of G (query heads divided by KV heads), enabling longer context windows without a proportional memory cost.

Extended RoPE base
: Phi-3, Phi-4, Gemma 3, and LLaMA 3.2 all use RoPE with extended base frequencies (θ = 500,000–1,000,000) to support context windows up to 128K tokens without architectural modification.

SwiGLU feed-forward network
: Replacing the standard two-layer feed-forward network (FFN) with a Swish-gated linear unit (SwiGLU) variant, adopted across nearly all post-2022 SLMs, gives a roughly 5% perplexity improvement at equal parameter count.

These changes, together with RoPE base extension and positional interpolation, have driven a rapid expansion of context windows in the SLM inventory (Figure 5): median supported context grew from a few thousand tokens in 2023 to over 100K by 2025.

3.1.2 Mixture-of-Experts (MoE) SLMs

MoE architectures [72, 73] decouple *total* parameters from *active* parameters per token: only a subset of "expert" feed-forward network (FFN) blocks is activated by a learned router. This allows a model to have a large total parameter count while keeping per-token FLOPs comparable to a much smaller dense model.

Mixtral 8×7B [74] (47B total, 12.9B active per token) is the canonical example of an MoE SLM-boundary model, competitive with 70B dense models while activating only 25% of parameters per token. More recent MoE SLMs include DeepSeek-MoE [75] (finer-grained experts, 2B active from 16B total) and Qwen3-30B-A3B [17], which activates only 3B of 30B parameters and reportedly matches Qwen2.5's dense base models at one-tenth the active parameter count. Meta's Llama 4 Scout [76] (109B total, 17B active, 16 experts) applies the same principle at the small-*active* scale, with Behemoth serving as the distillation teacher. Because deployment cost tracks *active*

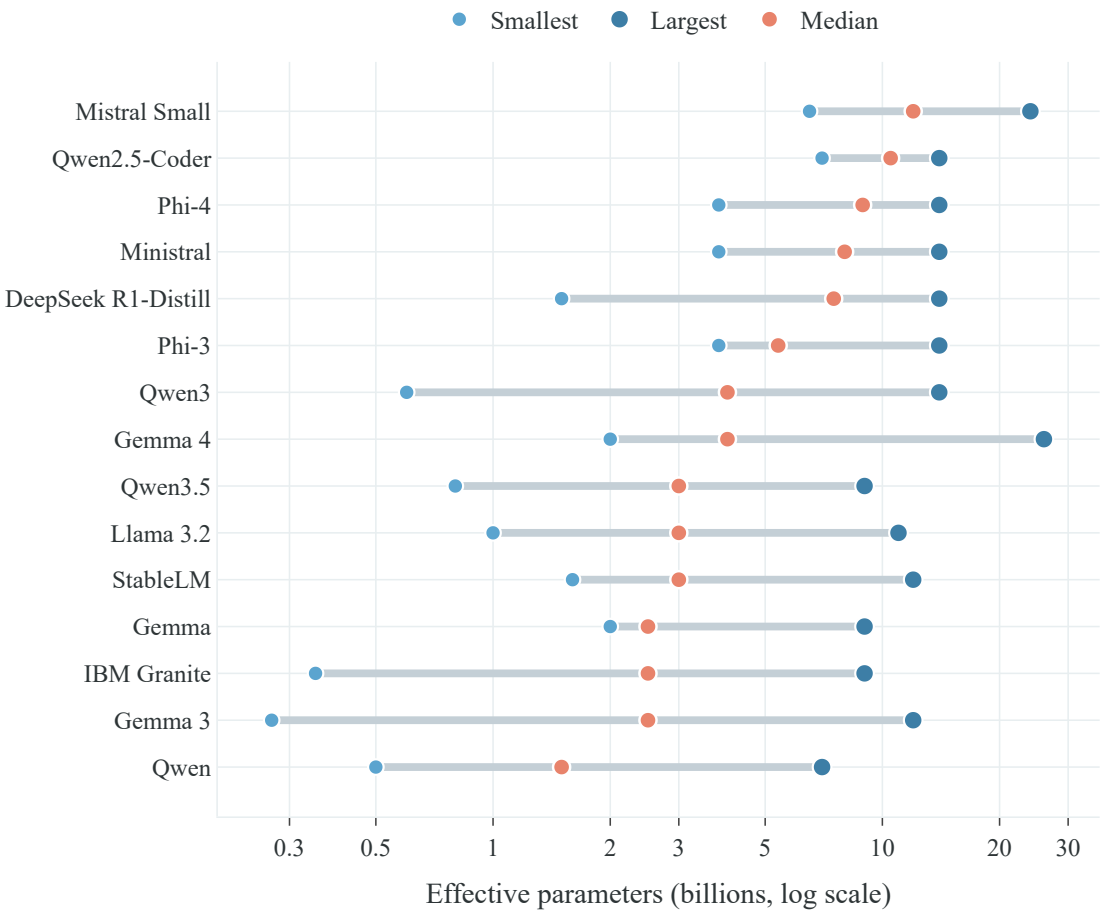


Figure 4. Parameter span (minimum, median, maximum, in billions, log scale) of the principal SLM families in the inventory.

rather than total parameters, several of these models fall inside the practical SLM envelope for latency and energy even though their storage footprint does not. Dense models still dominate the inventory numerically (Figure 6), but the MoE models that exist carry a much larger median total-parameter count at comparable per-token cost.

3.1.3 Hybrid SSM-Transformer Models

A nascent but rapidly growing class of SLMs combines attention-based transformer layers with state-space models (SSMs) [77], which process sequences in linear rather than quadratic time. Mamba [77] introduced hardware-aware selective SSMs; Jamba [78] interleaves transformer and Mamba blocks in a 1:7 ratio, achieving competitive performance with 2.6× lower KV-cache memory than a pure transformer. In 2025–2026 this design moved from research to production at the small scale. Falcon-H1 [16] spans 0.5B–34B with a parallel Mamba-2/attention block. IBM's Granite 4.0 [79] interleaves self-attention and Mamba-2 layers at roughly a 1:9 ratio and pairs it with a fine-grained MoE, reporting more than 70% memory reduction for long-context and multi-session inference relative to transformer-only models, with "Tiny" (~7B total, ~1B active) and "Micro" variants aimed explicitly at edge and local use. NVIDIA's Nemotron Nano 2 [80] replaces most self-attention in a 9–12B model with Mamba-2 layers (6 attention, 28 Mamba-2, 28 FFN across 62 layers) and reports up to 6× higher throughput than Qwen3-8B when generating long reasoning traces. Alibaba's Qwen3.5 small series adopts a Gated DeltaNet hybrid at a 3:1 linear-to-full-attention ratio [81]. Linear-time inference is most valuable exactly where SLMs are deployed—long-context, streaming, memory-bound edge settings—which is why the hybrid category is growing fastest.

3.2 Taxonomy by Training Strategy

3.2.1 Trained from Scratch

Most commercially significant SLMs are pre-trained from scratch on carefully curated datasets. The Phi family exemplifies a data-centric philosophy: rather than increasing parameters, Microsoft researchers found that training on "textbook-quality" synthetic and curated web data yields disproportionate gains [34, 82]. Phi-1 (1.3B) [34] trained on <8B tokens—two orders of magnitude less than contemporaries—yet achieved competitive Python

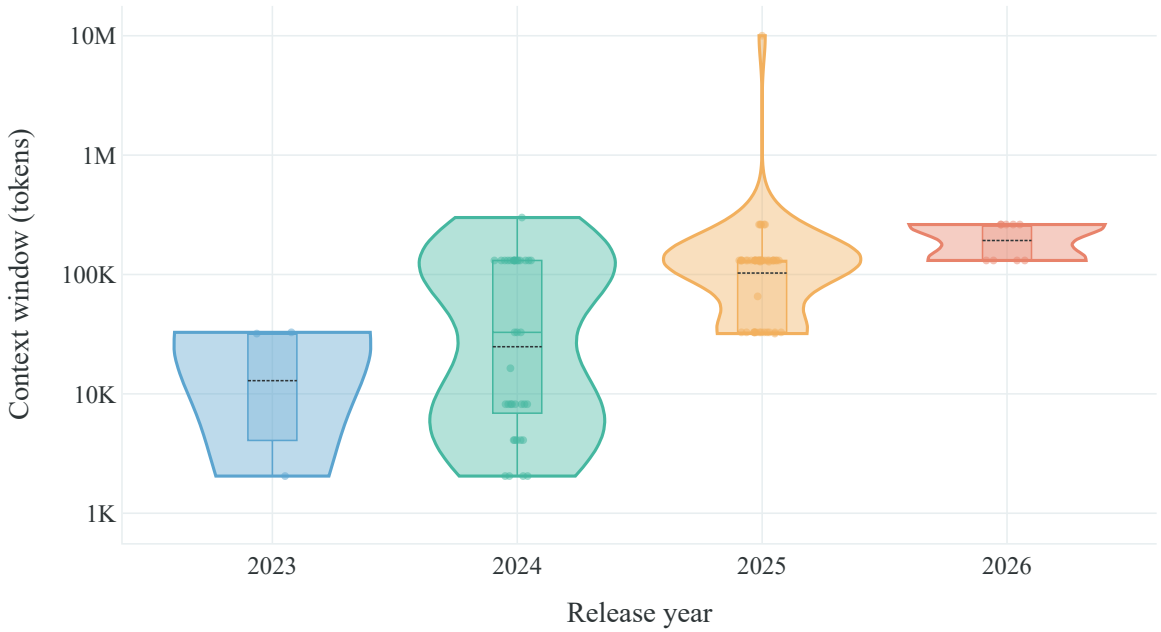


Figure 5. Supported context window of documented SLM variants by release year (log scale). Each violin is the kernel-density estimate of the distribution; the inner box marks the interquartile range, the dashed line the mean, and the dots individual models. Long context has become standard in the sub-10B regime.

coding benchmarks. LLaMA-3.2 3B [4] trained on 9T tokens from Meta's curated corpus. Gemma 3 [39] used 14T tokens with a curriculum that progressively increases the ratio of math, code, and multilingual data.

Key training innovations in this category include: (a) *data quality filtering* using heuristic and classifier-based pipelines; (b) *synthetic data generation* from larger teacher models [83, 84]; (c) *curriculum scheduling* where token mixture evolves over training to emphasize harder reasoning data in later stages; and (d) *long-context extension* via continued pre-training with positional interpolation or extended RoPE.

3.2.2 Distilled SLMs

Knowledge distillation [38] transfers capability from a large teacher model to a smaller student. In the LLM era, distillation takes several forms. *White-box distillation* accesses the teacher's internal representations (intermediate hidden states, attention maps), enabling feature-level alignment. *Black-box distillation* uses only the teacher's outputs (token logits or generated text) as supervision. Orca [84] demonstrated that distilling reasoning *traces* from GPT-4—rather than just answers—enables a 13B student to acquire substantially stronger chain-of-thought capabilities. Reasoning distillation became a dominant 2025 strategy: DeepSeek-R1-Distill models [85] transfer long chain-of-thought from DeepSeek-R1 into Qwen-2.5 students at 1.5B–14B, and Phi-4-mini-reasoning [86] combines distillation from a large reasoning teacher with reinforcement learning to bring competition-mathematics performance to a 3.8B model. Gemma 3 and Gemma 4 [18, 39] distill every model size, including the largest, from a bigger teacher by default.

3.2.3 Pruned and Compressed SLMs

Rather than training a small model from scratch, pruning removes parameters from a large pre-trained model. NVIDIA's Minitron pipeline [87] first prunes LLaMA-3.1-8B by removing attention heads, MLP neurons, and layers guided by activation statistics, then distills from the unpruned teacher, achieving a 40× reduction in training compute relative to training from scratch while matching or exceeding the performance of same-size models trained from scratch.

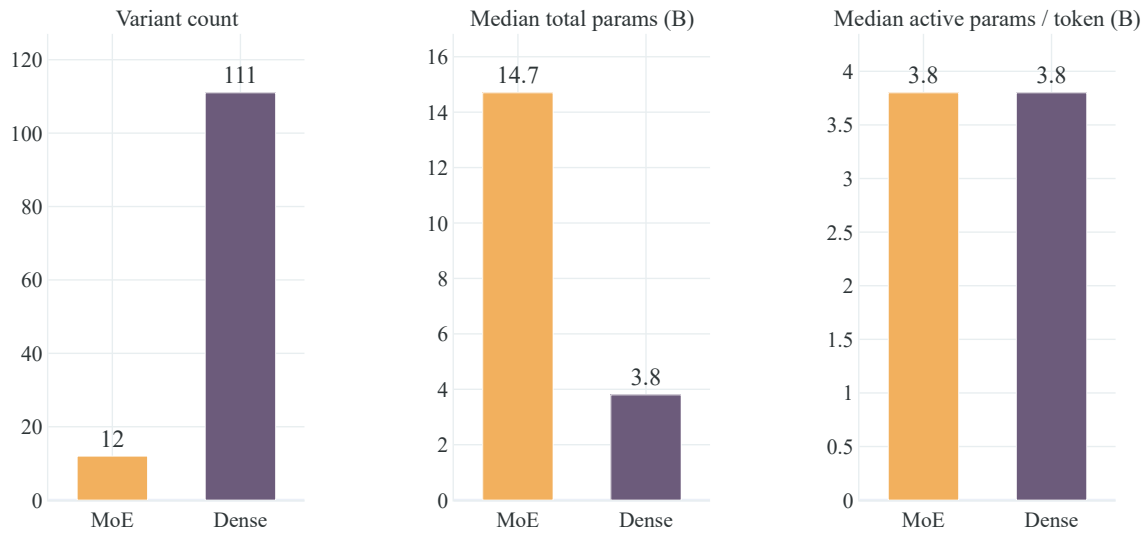


Figure 6. Mixture-of-experts versus dense SLMs in the inventory: variant count (left), median total parameters (centre), and median *active* parameters per token (right). MoE models carry a much larger total-parameter count but a per-token cost comparable to dense models.

3.3 Taxonomy by Modality

3.3.1 Text-Only SLMs

Text-only models still dominate the inventory (Figure 7), processing tokenized text as both input and output. This category encompasses all major SLM families: Phi, Gemma, LLaMA, Mistral, Qwen, Yi, InternLM, SmolLM, MiniCPM, OLMo, Pythia, and DeepSeek.

3.3.2 Multimodal (Vision-Language) SLMs

A growing subset accepts image inputs alongside text, enabling visual question answering, image captioning, document understanding, and visual reasoning. Notable examples include:

PaliGemma 3B [88]
: Google's 3B vision-language SLM combining a SigLIP-400M image encoder with a Gemma decoder; state-of-the-art on transfer benchmarks.

Phi-3-vision (4.2B)
: Microsoft's first vision-capable Phi model, adding a 400M CLIP encoder to Phi-3-mini's text backbone; competitive on DocVQA and ChartQA.

Gemma 3 (1B/4B/12B/27B)
: Google's Gemma 3 is the first SLM family where all sizes include image understanding by default, using a SigLIP image encoder [39].

Qwen2.5-VL 3B/7B
: Alibaba's multimodal SLMs with native support for video, charts, and documents.

3.3.3 Audio and Multi-Modal SLMs

A smaller but rapidly growing category incorporates audio alongside text and vision. Gemma 3n [89] is built mobile-first: its MatFormer ("Matryoshka") architecture nests a 2B-effective sub-model inside a 4B-effective model so that one download yields a spectrum of sizes, and Per-Layer Embeddings cut the resident memory footprint of the 5B/8B raw models to roughly that of a 2B/4B model (2–3 GB). Gemma 4 [18] extends the family from 2.3B-effective to 31B with dense and MoE variants, an integrated thinking mode, and, for the 12B model, an encoder-free architecture that ingests raw audio and image patches directly. Apple's on-device Foundation Model [19] is a ~3B multimodal model quantized with 2-bit quantization-aware training and KV-cache sharing. The full text+vision+audio frontier in a single sub-10B model is now a shipping capability rather than a research aspiration.

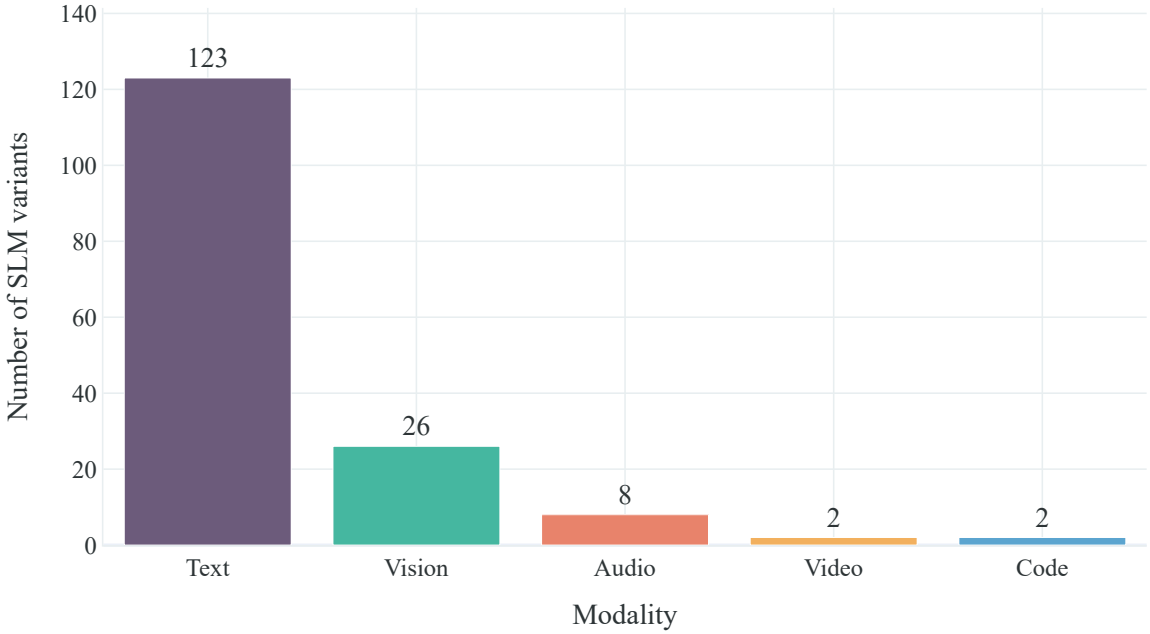


Figure 7. Input modalities supported by documented SLM variants (a variant may support several). Vision has become common; audio and video remain a small but growing minority.

3.4 Taxonomy by Application Domain

3.4.1 General-Purpose SLMs

General-purpose SLMs are trained on diverse data and evaluated on multi-domain benchmarks (MMLU, HellaSwag, ARC, GSM8K). The majority of SLMs in our taxonomy—particularly the flagship families from Microsoft (Phi), Google (Gemma), Meta (LLaMA), Mistral AI, Alibaba (Qwen), and DeepSeek—fall into this category.

3.4.2 Domain-Specific SLMs

Domain-specific SLMs sacrifice breadth for depth, outperforming general-purpose models on narrow tasks:

Healthcare
: Meerkat (7B) [90], fine-tuned on clinical QA datasets, surpasses GPT-4 on the USMLE Step 1 benchmark while running on a single A100 GPU. OptimCLM [91] applies combined compression (distillation + quantization + pruning) to clinical language models without sacrificing diagnostic accuracy.

Code
: Code LLaMA [92], StarCoder2 [93], and DeepSeek-Coder [94] represent a mature class of code-focused SLMs with strong HumanEval and MBPP performance.

Legal
: Domain-specific legal SLMs trained on corpora of statutory text, case law, and contracts reduce administrative processing time by 30–40% in pilot deployments while maintaining stricter data privacy than API-based LLMs.

Multilingual
: Aya 8B [95] covers 101 languages with strong cross-lingual instruction following. Qwen2.5 7B [14] serves as the de facto open-weight SLM for multilingual enterprise deployments.

Table 1 summarizes the four-dimensional taxonomy with representative models for each combination.

4 Model Compression Techniques for SLMs

Model compression is central to the SLM ecosystem: it is the primary pathway for reducing large pre-trained models to sizes deployable on edge hardware, and it provides a complementary route to training-from-scratch. The principal techniques are knowledge distillation, quantization, and pruning; in practice, compound pipelines combining all three have become the state of the art. Table 2 summarizes the key characteristics of each approach.

Table 1. Summary of the four-dimensional SLM taxonomy with representative models.

Dimension	Category	Key Characteristics	Representative Models
Architecture	Dense Transformer	All params active per token; GQA, RoPE, SwiGLU	Phi-4-mini, Qwen3, SmolLM3, Gemma 3
	MoE	Sparse activation; total ≫ active params	Mixtral 8×7B, Qwen3-30B-A3B, Llama 4 Scout
	Hybrid SSM-Transformer	Linear-time; interleaved SSM/attention blocks	Falcon-H1, Granite 4.0, Nemotron Nano 2
Training Strategy	From scratch	Curated/synthetic data; full pre-training	Phi, Qwen2.5/Qwen3, SmolLM3
	Distillation	Knowledge transfer from larger teacher	Gemma 3/4, DeepSeek-R1-Distill, Phi-4-mini-reasoning
	Pruning-based	Parameter removal from large model + distillation	Minitron, Sheared-LLaMA
Modality	Text-only	Tokenized text I/O; majority of production SLMs	Mistral 7B, SmolLM3, Qwen3
	Vision-Language	Image + text; CLIP/SigLIP encoder added	Phi-3-vision, Gemma 3, PaliGemma
	Audio/Multimodal	Text + vision + audio in one sub-10B model	Gemma 3n, Gemma 4, Apple AFM
Domain	General-purpose	Multi-domain training; evaluated on broad benchmarks	Phi, Gemma, LLaMA, Qwen, Mistral
	Healthcare	Clinical NLP; HIPAA-compliant on-premises deployment	Meerkat 7B, OptimCLM
	Code	Code corpora; FIM training; HumanEval/MBPP focus	Code LLaMA, StarCoder2, DS-Coder
	Legal/Multilingual	Domain corpora; regulatory and cross-lingual tasks	Aya 8B, Qwen2.5 multilingual

Table 2. Comparison of model compression techniques for SLMs.

Method	Requires Retraining	Compression Ratio	Accuracy Retention	Hardware Req.
Quantization (8-bit)	No	2×	~99%	Low
Quantization (4-bit)	No	4×	~96%	Low
Quantization (2-bit)	No	8×	~85%	Low
Pruning (50% sparse)	Partial	2×	~93%	Medium
Pruning (75% sparse)	Partial	4×	~82%	Medium
Structured Pruning	Yes	2–5×	~90%	Medium
Knowledge Distillation	Yes	10–100×	85–95%	High
KD + Pruning + Quant	Yes	40 × +	~90%	High

4.1 Knowledge Distillation

knowledge distillation (KD) [38] transfers the "dark knowledge" embedded in a teacher model's soft output distribution to a smaller student model. The student is trained to minimize a combination of the standard cross-entropy loss on labeled data and a Kullback–Leibler (KL) divergence loss with respect to the teacher's output logits at a temperature $T > 1$. This approach was pioneered for BERT compression (DistilBERT [10], TinyBERT [11]) and has since been substantially extended for generative SLMs.

4.1.1 White-Box vs. Black-Box KD

The survey by Yang et al. [96] classifies LLM distillation into two paradigms:

White-box KD has access to the teacher's intermediate representations. TinyBERT [11] pioneered layer-by-layer feature alignment, matching student hidden states, attention matrices, and embedding layers to teacher counterparts via MSE loss. In the decoder-only LLM setting, this requires pairing student and teacher layers—a non-trivial task when sizes differ substantially—and is computationally expensive but yields superior retention of teacher capabilities.

Black-box KD uses only the teacher's generated outputs—either token probabilities or free-form text—as the supervision signal. Orca [84] exemplifies text-output black-box KD: GPT-4 generates step-by-step reasoning traces, which a LLaMA-based student learns to replicate. DeepSeek-R1 [85] extends this to long chain-of-thought reasoning distillation, producing 7B and 14B students that dramatically outperform identically-sized from-scratch models on mathematical reasoning.

4.1.2 Chain-of-Thought Distillation

A particularly productive form of black-box KD is chain of thought (CoT) distillation [97]: the teacher model generates explicit reasoning steps, and the student learns to produce or internalize these steps. Feng et al. [98] demonstrate counterfactual distillation—augmenting CoT traces with contrastive incorrect reasoning—as a way to sharpen the student's reasoning boundaries. Liao et al. [99] propose Neural-Symbolic Collaborative Distillation, using a neuro-symbolic teacher to provide structured reasoning supervision that is more transferable to small students than purely neural CoT.

4.1.3 Data Augmentation in KD

The survey by Xu et al. [100] highlights data augmentation (DA) as a central paradigm within KD frameworks. When the student lacks sufficient coverage of the teacher's knowledge space, the teacher can generate synthetic training instances targeting specific skill gaps—the mechanism underlying the Phi family's success [34]. The efficient KD approach of Ballout et al. [101] demonstrates that targeted data augmentation can reduce the student training compute by 60% while preserving 95% of the teacher's task-specific accuracy.

4.1.4 Self-Distillation

An emerging approach eliminates the external teacher entirely. Fu et al. [102] propose Dynamic Self-Distillation, in which the model uses its own predictions from previous mini-batches as soft targets during fine-tuning, providing a form of implicit regularization that reduces overfitting on small fine-tuning datasets—a critical challenge for SLMs.

4.2 Quantization

Quantization reduces the numerical precision of model weights and/or activations from 32-bit floats (FP32) or 16-bit (FP16/BF16) to integers or low-bit floats (INT8, INT4, FP4, NF4, INT2). This directly reduces memory footprint and, on hardware with native integer arithmetic (NPUs, modern CPUs), reduces inference latency.

4.2.1 Post-Training Quantization (PTQ)

PTQ is performed after training is complete, requiring only a small calibration dataset (typically 128–512 samples) and no gradient computation. It is the dominant quantization paradigm because it is cheap and applies to any model without access to training infrastructure.

GPTQ [43] applies second-order weight compression: it uses an approximation of the inverse Hessian of the loss with respect to each layer's weights to determine optimal rounding decisions, minimizing the layer-wise squared error introduced by quantization. For SLMs, GPTQ 4-bit quantization of a 7B model reduces video memory (VRAM) from ~14 GB to ~4 GB.

AWQ [44] observes that not all weights are equally important for activation output: 1% of weights (those corresponding to large-magnitude activation channels) account for disproportionate reconstruction error. AWQ identifies these "salient" channels using a small calibration set and applies per-channel scaling to protect them before quantization, consistently outperforming GPTQ on complex reasoning tasks and winning the MLSys 2024 Best Paper award.

GGUF is a quantization file format popularized by `llama.cpp` [45] that packages model weights, vocabulary, and architecture metadata in a single binary file. It supports a range of quantization levels (Q2_K through Q8_0), letting users trade off quality against memory footprint along a near-continuous spectrum, and it enables Llama-3.2-3B to run at 60+ tokens per second on consumer laptops without a GPU.

SqueezeLLM [46] introduced sensitivity-based non-uniform quantization with dense-and-sparse decomposition, showing that sensitivity varies dramatically across weight dimensions and enabling higher accuracy at ultra-low bit-widths.

4.2.2 Quantization-Aware Training (QAT)

QAT integrates simulated quantization into the training loop, allowing the model to learn weight values that are robust to quantization error. This is particularly valuable for aggressive quantization (2-bit, 3-bit) where PTQ accuracy drops unacceptably. QAT produces models with 2–3% higher task accuracy than PTQ at the same bit-width but requires access to training data and significant compute.

4.2.3 Quantization versus Pruning in the Sub-10B Regime

Kurz and Zhao [26] conducted the first systematic comparison of PTQ quantization (GPTQ, AWQ) against post-training pruning (SparseGPT, Wanda) specifically in the SLM regime (0.5B–3.8B parameters, six models, seven languages, seven downstream tasks). Their primary conclusion: *quantization consistently outperforms pruning in preserving model fidelity, multilingual perplexity, and reasoning accuracy*. AWQ at 4-bit outperforms 50% Wanda pruning on 6 of 7 tasks. Quantization's advantage is most pronounced on language modeling and multilingual tasks; the gap narrows on complex knowledge tasks, where both methods show similar degradation under high compression.

4.3 Pruning

Pruning removes parameters from a pre-trained model, either individually (unstructured) or in structured groups (entire heads, layers, or channels). Structured pruning yields hardware-friendly sparse models compatible with

standard matrix multiplication libraries; unstructured pruning achieves higher compression ratios but requires special sparse hardware or software to realize latency benefits.

4.3.1 Unstructured Pruning

SparseGPT [47] extends the GPTQ framework to weight removal: it prunes weights layer-by-layer using inverse-Hessian information to minimize the reconstruction error introduced by zeroing each weight. It can prune the 175B GPT-3 to 50% sparsity in a single forward pass without weight updates, with a perplexity increase below 1%.

Pruning by weights and activations (Wanda) [103] proposes a simpler criterion: remove the weights with the smallest product of weight magnitude and corresponding input-activation norm. This needs only a forward pass over the calibration data (no second-order information) and runs about 10× faster than SparseGPT while matching its accuracy at 50% sparsity.

4.3.2 Structured Pruning

Structured pruning removes entire attention heads, MLP channels, or transformer layers, yielding immediately deployable models without special sparsity libraries. Layer pruning—removing the least important layers as measured by cosine similarity between adjacent layer outputs—has been shown to reduce LLaMA-3 8B to a 6B model with < 2% MMLU degradation. The Minitron pipeline [87] applies depth and width pruning simultaneously, guided by importance scores derived from activation magnitudes over a calibration set.

4.4 Combined Compression Pipelines

The state of the art in SLM compression combines pruning, distillation, and quantization in a multi-stage pipeline. NVIDIA Minitron [87] achieves a 40× reduction in training compute relative to training LLaMA-3.1-8B from scratch by: (1) structured pruning to target size; (2) knowledge distillation from the original 8B teacher to the pruned student; (3) post-training quantization for deployment. The resulting 4B Minitron model matches the performance of 4B models trained from scratch on commonsense, code, and math benchmarks.

OptimCLM [91] applies a similar pipeline to clinical language models, combining all three compression stages to produce models under 1B that achieve within 5% of clinical NLP accuracy of their 7B counterparts. These results suggest that the combination of pruning (for architectural reduction) + distillation (for capability recovery) + quantization (for memory reduction) represents the practical frontier of SLM compression.

5 Consolidated Reported Performance

This section assembles a comparative picture of SLM capability and deployment behavior from the public record. In contrast to an original benchmarking study, the performance values analyzed here are *reported* figures, synthesized from vendor technical reports, official model cards, community leaderboards, and third-party evaluations. We first document the protocol used to collect and reconcile these figures (Section 5.1), then present the consolidated synthesis (Sections 5.2–5.7). Every table entry traces to a dated, citable origin recorded in Appendix A; where sources disagree, the reconciliation rule of Section 5.1.4 applies.

5.1 Data-Collection Protocol

5.1.1 Model Selection

Model inclusion required: (1) public availability of weights or a publicly documented evaluation card; (2) parameter count below 10 billion, with Phi-4 at 14B retained as an upper-bound reference; (3) at least one reported score on an established benchmark used here; and (4) representation of diverse training methodologies (synthetic-data-centric, web-scale pretraining, distillation). Table 3 lists the resulting 17-model cohort with architectural specifications as reported by each model's release materials. The cohort spans four orders of magnitude in parameter count and six developer organizations, enabling analysis of how training methodology, architecture, and scale interact to determine reported performance.

5.1.2 Benchmark Tasks and Metrics

Fifteen benchmark tasks across seven NLP categories were included: MMLU [104], HellaSwag [105], Winogrande [106], ARC-Challenge [107], PIQA [108], and BoolQ [109] for commonsense and language reasoning; GSM8K [110] and MATH [111] for mathematical reasoning; HumanEval [112] and MBPP [113] for code generation; MNLI [114] and ANLI [115] for natural language inference; TriviaQA [116] and SQuAD 2.0 [117] for question answering; CNN/DailyMail [118] and XSum [119] for summarization; and MT-Bench [120] for instruction following. Inclusion of a task required that at least three reviewed models have publicly reported scores under compatible settings. Reported figures correspond to each benchmark's standard metric (accuracy for multiple choice; EM/F1 for QA; pass@1 for code; ROUGE-L for summarization; GPT-4 judge score for MT-Bench; exact match for GSM8K; symbolic

Table 3. Architectural specifications of reviewed models (as reported by each model's official release materials).

Model	Params	Layers	Hidden	Attention	Context	Vocab	Train Tokens
TinyLlama-1.1B	1.1B	22	2,048	MHA-32h	2K	32K	3.0T
SmolLM2-1.7B	1.7B	24	2,048	MHA-32h	2K	49K	11.0T
Qwen2.5-0.5B	0.5B	24	896	GQA-14/2	32K	152K	18.0T
Qwen2.5-1.5B	1.5B	28	1,536	GQA-12/2	32K	152K	18.0T
Qwen2.5-3B	3.0B	36	2,048	GQA-16/2	32K	152K	18.0T
Qwen2.5-7B	7.0B	28	3,584	GQA-28/4	128K	152K	18.0T
Llama 3.2-1B	1.2B	16	2,048	GQA-32/8	128K	128K	9.0T
Llama 3.2-3B	3.2B	28	3,072	GQA-24/8	128K	128K	9.0T
Gemma 2-2B	2.6B	26	2,304	GQA-8/4	8K	256K	2.0T
Gemma 2-9B	9.2B	42	3,584	GQA-16/8	8K	256K	8.0T
Gemma 3-1B	1.0B	26	1,536	GQA-LG	32K	262K	2.0T
Gemma 3-4B	4.3B	34	2,560	GQA-LG	128K	262K	4.0T
Mistral 7B	7.3B	32	4,096	GQA-32/8	8K	32K	—
Phi-3-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.3T
Phi-3.5-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.4T
Phi-4-mini	3.8B	32	3,072	GQA-24/8	128K	200K	5.0T
Phi-4	14.0B	40	5,120	GQA	16K	100K	~10T

Note: MHA = Multi-Head Attention; GQA = Grouped-Query Attention (Q/KV heads); GQA-LG = GQA with local–global interleaving. Training-token counts are reported or estimated values from each model's official technical report or model card.

equivalence for MATH). Efficiency metrics—tokens per second, time-to-first-token, peak memory, on-disk size, and reported energy—are each annotated in Appendix A with the measurement conditions given by their source, because these numbers are especially sensitive to hardware, batch size, context length, and backend.

5.1.3 Hardware and Quantization Scope

The review covers reported behavior across six configurations spanning the cloud-to-edge spectrum (Table 4) and three precisions: FP16 (reference), INT8, and INT4, with GGUF Q4_K_M, GPTQ group-128, and AWQ group-128 treated as distinct INT4 subclasses when the source distinguishes them. Inference-framework attribution (vLLM [121], llama.cpp [45], ONNX Runtime [63], ExecuTorch [61], MLC-LLM [62]) follows the stack used by the source of each figure; cross-framework comparisons are made only when compared figures share a framework or are explicitly flagged.

Table 4. Hardware configurations covered by the review. For each reviewed model, reported performance on a configuration was included where publicly available.

Configuration	Hardware	RAM/VRAM	TDP	Use Case
Cloud GPU	NVIDIA A100 80GB	80 GB HBM2e	300W	Training + serving
High-end Consumer	NVIDIA RTX 4090	24 GB GDDR6X	450W	Local inference
Mid-range Consumer	NVIDIA RTX 3060	12 GB GDDR6	170W	Budget GPU
CPU-only (Laptop)	Intel i7-13700H	32 GB DDR5	45W	Laptop deployment
Edge SBC	Raspberry Pi 5	8 GB LPDDR4X	15W	Embedded IoT
Mobile SoC	Snapdragon 8 Gen 3	12 GB LPDDR5X	~10W	Smartphone

5.1.4 Source Selection, Extraction, and Reconciliation

Reported values were collected in decreasing order of priority from: (1) official vendor technical reports and release papers; (2) official model cards on Hugging Face and vendor repositories, including linked evaluation appendices; (3) community-operated standardized evaluations (the Open LLM Leaderboard, HELM [122], and lm-evaluation-harness [123]-based community reports); (4) third-party independent evaluations (Artificial Analysis, LMSYS Chatbot Arena, peer-reviewed follow-up studies); and (5) vendor developer blogs, used only to fill gaps. All figures were extracted within a single collection window to preserve time-consistency across models.

When qualifying sources disagreed on the same model–task–condition tuple, the figure from the source higher in the priority list was retained and the alternative recorded in Appendix A; where the discrepancy exceeded 2.0 percentage points (or the metric equivalent), both figures were retained as a reported range or the tuple was excluded. A tuple was excluded from cross-model comparison when: (a) only a single self-reported value existed

Table 5. Reported zero-shot FP16 benchmark scores (instruction-tuned variants). **Bold** = best among SLMs. All values are reported figures; per-figure sources in Appendix A. MATH and MT-Bench columns and the extended task set appear in Table 15.

Model	Params	ARC-C	HellaSwag	GSM8K	HumanEval	MMLU
Phi-4-mini-Inst.	3.8B	83.7	69.1	**88.6**	72.0	67.3
Qwen2.5-7B-Inst.	7.6B	**90.1**	80.0	88.7	**84.8**	**72.6**
Gemma 2-9B-Inst.	9.2B	89.8	**87.1**	84.9	68.9	71.3
Gemma 3-4B-Inst.	4.0B	85.2	78.4	82.1	64.6	65.8
LLaMA 3.2-3B-Inst.	3.2B	76.1	77.2	75.6	58.5	61.8
LLaMA 3.2-1B-Inst.	1.2B	62.3	63.8	47.2	35.4	48.6
Mistral 7B-Inst. v0.3	7.3B	82.7	73.5	81.9	66.5	63.0
Qwen2.5-3B-Inst.	3.0B	78.6	72.1	79.8	65.2	65.6
Qwen2.5-1.5B-Inst.	1.5B	68.4	64.2	61.8	42.8	58.3
Qwen2.5-0.5B-Inst.	0.5B	52.1	48.3	38.5	22.0	47.5
SmolLM2-1.7B-Inst.	1.7B	59.8	60.2	42.6	28.7	50.1
TinyLlama-1.1B	1.1B	47.5	53.8	18.4	12.2	36.2
StableLM 2-1.6B	1.6B	55.2	57.6	32.1	22.5	44.3
GPT-4o (reference)	—	*96.4*	*95.3*	*95.8*	*90.2*	*88.7*
LLaMA 3.1-70B-Inst.	*70B*	*92.8*	*88.5*	*93.2*	*80.5*	*82.2*

and could not be cross-checked; (b) the harness, prompt format, or shot count materially differed from the setting used by at least half the compared models and could not be normalized; or (c) the source did not specify base versus instruction-tuned status.

5.1.5 Limitations of the Evidence Base

Because the figures are compiled from heterogeneous public sources rather than a single controlled pipeline, four limitations bound the strength of every comparison below. **Prompt-format and harness variation:** different sources use different templates and answer-extraction heuristics, which can shift reported accuracy by several points under the same nominal benchmark. **Self-reporting bias:** vendor-reported scores are systematically more favorable than third-party reproductions; where third-party figures exist, comparisons preferentially use them and flag the divergence. **Condition-coverage sparsity:** not every model has reported scores for every task, tier, or precision; sparse tuples are marked, not imputed. **Snapshot staleness:** the ecosystem evolves monthly, and figures reflect the state of public reporting at the collection window. These limitations motivate the empirical-verification agenda of Section 10.7.

5.2 Overall Benchmark Performance

Several patterns are consistent across the sources reviewed (Table 5). No single SLM leads on every task. In the 3–4B band, Phi-4-mini is reported strongest on mathematical reasoning (GSM8K) and MATH, while Qwen2.5-3B is close behind with broader consistency. In the 7–10B band, Qwen2.5-7B is reported ahead of Mistral 7B and Gemma 2-9B on knowledge and code benchmarks. Phi-4 (14B) serves as an upper-bound reference. The most striking regularity is the effect of training data: models of near-identical size but different training corpora—Qwen2.5-1.5B versus SmolLM2-1.7B, or Qwen2.5-0.5B versus older 500M-class models—differ by 15–25 points on knowledge benchmarks, a gap far larger than any attributable to architecture at this scale. A Pareto view of reported MMLU against parameter count places Phi-4-mini and Qwen2.5-3B on the sub-4B efficient frontier and Qwen2.5-7B and Gemma 2-9B on the 7–10B frontier.

The extended task set (Appendix A.6, Table 15) adds ARC-E, TriviaQA, SQuAD 2.0, MBPP, SST-2, and MNLI and does not change these orderings.

5.3 Fine-Tuning versus Prompting

Across sources that report all three paradigms for a common model–task pair (Table 6), QLoRA adaptation yields the largest gains for the smallest models and for tasks where the base model is weak: reported sub-3B gains average roughly 8–10 percentage points over zero-shot, against roughly 4–5 points at 7B. Few-shot prompting is reported to add a smaller, consistent 1.5–4 points but consumes context and raises latency, making it less attractive for on-device agents. The practical reading, and the one most relevant to Section 7, is a specialization threshold near 3B: below it, task-specific fine-tuning is close to mandatory for delegated agent sub-tasks; above it, few-shot prompting is often sufficient for well-represented tasks.

Table 6. Reported zero-shot, 5-shot, and QLoRA fine-tuned accuracy for selected model–task pairs. Values are reported figures.

Model	Setting	SST-2	MNLI	ARC-C	GSM8K	SQuAD 2.0 (F1)
Phi-4-mini	Zero-shot	92.4	72.1	83.7	88.6	74.2
	5-shot	94.1	75.8	85.2	89.3	77.8
	QLoRA	**96.8**	**82.4**	86.1	89.8	**83.6**
Qwen2.5-7B	Zero-shot	93.8	74.6	90.1	88.7	76.8
	5-shot	95.2	77.3	91.2	89.5	79.4
	QLoRA	**97.3**	**84.1**	91.5	90.1	**85.2**
LLaMA 3.2-3B	Zero-shot	88.2	63.5	76.1	75.6	65.4
	5-shot	91.5	68.2	78.8	77.4	70.2
	QLoRA	**95.4**	**78.6**	80.3	79.8	**78.5**
SmolLM2-1.7B	Zero-shot	82.6	54.2	59.8	42.6	52.3
	5-shot	86.4	59.8	63.1	44.8	57.6
	QLoRA	**93.2**	**72.5**	65.8	48.2	**68.4**

5.4 Quantization Impact

Table 7. Reported effect of INT4 quantization on benchmark accuracy. Subscripts show the reported percentage-point change from the FP16 baseline.

Model	Precision	ARC-C	HellaSwag	GSM8K	MMLU	Avg. Retention
Qwen2.5-7B	FP16	90.1	80.0	88.7	72.6	100.0%
	AWQ-4bit	$88.9_{-1.2}$	$79.5_{-0.5}$	$87.2_{-1.5}$	$71.8_{-0.8}$	98.8%
	Q4_K_M	$88.1_{-2.0}$	$78.8_{-1.2}$	$86.1_{-2.6}$	$70.9_{-1.7}$	97.7%
Phi-4-mini	FP16	83.7	69.1	88.6	67.3	100.0%
	AWQ-4bit	$82.6_{-1.1}$	$68.6_{-0.5}$	$86.9_{-1.7}$	$66.5_{-0.8}$	98.6%
	Q4_K_M	$81.8_{-1.9}$	$67.9_{-1.2}$	$85.8_{-2.8}$	$65.8_{-1.5}$	97.6%
LLaMA 3.2-3B	FP16	76.1	77.2	75.6	61.8	100.0%
	Q4_K_M	$73.2_{-2.9}$	$75.4_{-1.8}$	$71.8_{-3.8}$	$59.6_{-2.2}$	96.3%
SmolLM2-1.7B	FP16	59.8	60.2	42.6	50.1	100.0%
	Q4_K_M	$55.4_{-4.4}$	$56.8_{-3.4}$	$37.2_{-5.4}$	$46.5_{-3.6}$	92.4%

The sources converge on three points (Table 7). INT8 is reported as effectively lossless (sub-point degradation on knowledge and math benchmarks). INT4 costs roughly 1–4 points on average, and the loss is *not* uniform: smaller models degrade more, consistent with the argument that each weight in a small model carries more information [124]. Mathematical reasoning is reported to degrade faster than knowledge recall under INT4, by a factor of roughly 1.7–2.2. Among INT4 methods, AWQ is reported to retain slightly more accuracy than GPTQ for SLMs [44], and GGUF's Q4_K_M offers the best size–quality balance for CPU and edge backends. Reported model-size reductions are close to the theoretical 4×: a 7B model drops from roughly 15 GB to roughly 4.5 GB, and a 3.8B model to roughly 2.4 GB (Table 8).

Table 8. Reported model-weight memory footprint by precision (GB).

Model	FP16	GPTQ-4bit	AWQ-4bit	Q4_K_M
Qwen2.5-7B	15.2	4.2	4.4	4.5
Phi-4-mini	7.6	2.2	2.3	2.4
LLaMA 3.2-3B	6.4	1.9	2.0	2.1
SmolLM2-1.7B	3.4	1.1	1.1	1.2
TinyLlama-1.1B	2.2	0.7	0.8	0.8

5.5 Edge Deployment

The reported edge figures (Table 9) draw clear boundaries. An 8 GB Raspberry Pi 5 is reported to run INT4 models up to about 3B at marginally interactive speeds and sub-2B models comfortably. The Jetson Orin Nano handles

Table 9. Reported edge-deployment figures (INT4/Q4_K_M unless noted). Status labels indicate reported practical interactivity on the tested device.

Model	Device	tok/s	TTFT (ms)	Peak RAM (GB)	Load (s)	Status
SmolLM2-1.7B	Raspberry Pi 5	8.2	890	1.4	4.2	Good
TinyLlama-1.1B	Raspberry Pi 5	10.5	680	1.0	3.1	Good
LLaMA 3.2-1B	Raspberry Pi 5	9.8	720	1.1	3.4	Good
LLaMA 3.2-3B	Raspberry Pi 5	4.8	1,450	2.3	7.8	Marginal
Phi-4-mini	Raspberry Pi 5	3.2	2,100	2.6	9.5	Marginal
Qwen2.5-7B	Raspberry Pi 5	1.2	5,800	5.1	22.4	No
SmolLM2-1.7B	Jetson Orin Nano	42.5	125	1.6	2.8	Good
TinyLlama-1.1B	Jetson Orin Nano	54.2	98	1.2	2.1	Good
LLaMA 3.2-3B	Jetson Orin Nano	28.4	210	2.5	4.5	Good
Phi-4-mini	Jetson Orin Nano	22.8	285	2.8	5.2	Good
Qwen2.5-7B	Jetson Orin Nano	11.5	620	5.3	12.8	Marginal
Gemma 2-9B	Jetson Orin Nano	6.2	1,150	6.4	18.5	No
LLaMA 3.2-1B*	Galaxy S24+ (NPU)	45.2	85	1.2	3.8	Good
LLaMA 3.2-3B*	Galaxy S24+ (NPU)	22.5	165	2.4	6.2	Good
Phi-4-mini	Galaxy S24+ (CPU)	12.8	380	3.0	8.5	Good
Qwen2.5-7B	Galaxy S24+ (CPU)	5.4	920	5.5	18.2	Marginal

*Reported with a 4-bit NPU backend (ExecuTorch/SpinQuant).

everything up to 9B at useful speeds. Snapdragon-class smartphones with NPU delegation are reported to run models up to 7B, with interactive use practical below 4B. Time-to-first-token, dominated by prefill, is the binding constraint for conversational use: reported time to first token (TTFT) for a 3B INT4 model is around 4 s on the Pi but around 1 s on the Jetson. GQA and local–global attention substantially reduce the reported KV-cache footprint, which otherwise dominates memory at long context.

5.6 Cost and Energy

Table 10. Reported energy-per-inference estimates (joules per 1,000 completions, 512-token generation).

Model	A100	RTX 4090	Laptop CPU	Jetson	RPi 5
Qwen2.5-0.5B	180	320	410	280	612
Qwen2.5-3B	420	680	1,250	580	2,760
Phi-4-mini	510	780	1,480	650	3,180
Qwen2.5-7B	720	1,100	2,340	1,020	—
Phi-4 (14B)	1,350	1,850	—	—	—

Reported energy figures (Table 10) show that a datacenter GPU, despite its high power draw, achieves the lowest energy *per inference* for any given model because of its throughput; an edge board consumes more energy per inference but roughly 20× less absolute power, which favors it for always-on, low-duty-cycle workloads. Normalizing accuracy by energy, a small model on cheap hardware delivers markedly more benchmark points per joule than a large model on a datacenter GPU, provided its accuracy clears the task threshold. Table 12 in Section 6 translates these into cost-per-token comparisons.

5.7 Multilingual and Safety Reporting

Reported multilingual scores (FLORES-200, XQuAD, MGSM) place Qwen2.5-7B ahead of other SLMs, reflecting its large multilingual training corpus, with a wide gap to English-centric models on non-English mathematical reasoning. Reported toxicity evaluations (ToxiGen, RealToxicityPrompts) show substantially higher toxicity for the smallest models (TinyLlama, sub-1B variants), consistent with the argument that safety alignment is harder to preserve at reduced scale and is often absent from research-grade small models. Both observations carry directly into Section 7: an agent that routes non-English or safety-sensitive calls to a small English-centric model inherits these gaps, which is an argument for capability-aware rather than size-only routing.

6 Deployment on Resource-Constrained Environments

6.1 On-Device and Mobile Deployment

On-device deployment eliminates the latency, cost, and privacy exposure of cloud API inference. It enables AI functionality in offline and low-bandwidth environments and is a prerequisite for truly private personal assistants. The primary challenges are memory (RAM and storage), sustained inference throughput under thermal constraints, and battery impact.

6.1.1 Apple Intelligence (AFM)

Apple's Foundation Models [19, 125] are the most fully documented on-device SLM deployment. The 2025 report [19] describes a ~3B on-device model, optimized for Apple silicon through KV-cache sharing (dividing the model so the second block reuses the first block's cache, a reported 37.5% memory saving) and 2-bit quantization-aware training, paired with a Parallel-Track Mixture-of-Experts server model on Private Cloud Compute. The on-device model is reported competitive with Qwen3-4B and Gemma-3-4B on its target tasks; the server model with Llama 4 Scout. The third-generation framework [126] adds a natively multimodal, sparsely activated on-device model and opens the same API surface to third-party cloud models, so that an application calls the local SLM by default and escalates to a frontier LLM only on demand. Google's Gemini Nano and Samsung's Gauss [127] occupy the same design point on Android and Galaxy devices.

6.1.2 Samsung Gauss

Samsung Gauss [127] is a family of language models for Galaxy devices, including a ~1.7B on-device model targeting the Gauss1 chip and Exynos 2400 NPU. Samsung reports that Gauss enables on-device email summarization, translation, and writing assistance without requiring internet connectivity.

6.1.3 General Mobile Deployment Considerations

SLMs targeting mobile deployment must navigate three constraints. (1) *Memory*: Consumer smartphones typically provide 6–12 GB RAM, of which the OS uses 2–4 GB, leaving 4–8 GB for applications. A 3B model at Q4 requires ~2 GB, comfortable for most flagship devices. (2) *Thermal*: Sustained inference on mobile NPUs is throttled by thermals; realistic sustained throughput is lower than peak throughput benchmarks suggest. (3) *Battery*: Gemma 3 on Android reportedly consumes ~0.75% battery per prompt on Pixel 9 Pro, enabling hundreds of interactions per charge. The fundamental constraint in edge deployment is memory bandwidth rather than compute: mobile devices offer 50–90 GB/s of memory bandwidth versus 2–3 TB/s for datacenter GPUs [66], making INT4 quantization not merely a storage optimization but a critical performance enabler.

6.2 Edge and IoT Deployment

Below smartphone-class hardware lies a diverse ecosystem of edge devices: Raspberry Pi single-board computers, NVIDIA Jetson modules, and microcontrollers. SLMs targeting this tier typically require:

Sub-1B or extreme quantization
: A Raspberry Pi 5 (8 GB RAM, Cortex-A76 CPU) can run Q4 quantized 1B models at 10–20 tokens/second. SmolLM2-135M at Q8 runs at >30 tok/s on Raspberry Pi 5. Benchmarking shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models on single-board computers [60].

Jetson Orin
: NVIDIA's Jetson Orin NX (16 GB) with TensorRT-LLM can serve 7B quantized models at ~30 tok/s, enabling real-time local inference for industrial and robotics applications.

Microcontrollers (MCU)
: Sub-100M models using aggressive quantization (INT4/INT2) can run on ARM Cortex-M series MCUs with 512 KB–2 MB SRAM for specialized tasks like keyword detection and sensor data analysis.

6.3 Deployment Frameworks

A mature ecosystem of open-source frameworks enables SLM deployment across hardware targets. Table 11 provides a structured comparison.

6.4 Latency Comparison: SLMs vs. LLMs

Table 12 illustrates the practical latency implications of SLM vs. LLM deployment for three representative real-world scenarios.

Table 11. Key SLM deployment frameworks, their primary targets, and characteristics.

Framework	Primary Target	Quantization	Notes
llama.cpp [45]	CPU, Metal, CUDA, ROCm	GGUF (Q2–Q8)	De facto standard; minimal dependencies; ecosystem includes Ollama, llamafile
ONNX Runtime [63]	GPU, CPU, NPU, WASM	INT4/INT8	Cross-platform; DirectML/CoreML/CUDA backends; Phi-3-mini at 35 tok/s on Snapdragon NPU
MLX [128]	Apple Silicon	4-bit	Unified memory; eliminates GPU transfers; 55+ tok/s for LLaMA-3.2-3B on M3 Pro
TensorRT-LLM [129]	NVIDIA GPUs	INT4/INT8/FP16	Continuous batching; kernel fusion; 2–3× over PyTorch for 7B models
ExecuTorch [61]	Mobile (iOS, Android)	INT4	50 KB base runtime; 12+ hardware backends; deployed in Instagram, WhatsApp
MLC-LLM [62]	iOS, Android, web, MCU	4-bit	TVM compilation; WebGPU support; universal platform coverage
WebLLM [64]	Web browsers	4-bit WASM	WebGPU acceleration; 80% of native performance; OpenAI-compatible API

Table 12. Latency and cost comparison: SLMs vs. LLMs for representative use cases.

Metric	Real-time Chatbot	Code Completion	Edge IoT
SLM (3B, Q4, consumer GPU)			
TTFT	80–150 ms	30–80 ms	50–120 ms
Throughput	200–400 tok/s	300–500 tok/s	100–300 tok/s
Memory	~2 GB	~2 GB	~1 GB
Cost/1M tokens	$0.05–0.2	$0.05–0.2	$0 (on-device)
LLM (70B, FP16, multi-GPU)			
TTFT	500–2000 ms	200–800 ms	Not feasible
Throughput	30–80 tok/s	60–150 tok/s	N/A
Memory	~140 GB	~140 GB	N/A
Cost/1M tokens	$3–10	$3–10	N/A
Frontier API (GPT-4o, cloud)			
TTFT	300–1500 ms	200–600 ms	Requires internet
Throughput	80–200 tok/s	80–200 tok/s	N/A
Cost/1M tokens	$5–15	$5–15	N/A

The data in Table 12 illustrate a 5–20× cost advantage and comparable or superior latency for SLMs in chatbot and code-completion scenarios, with an absolute deployment advantage in offline edge contexts where LLMs are simply infeasible.

7 Small Language Models as the Substrate of Agentic AI

The preceding sections establish three facts about the sub-10B regime: modern SLMs reach capability levels that were state-of-the-art for far larger models only two years earlier (Section 5); they can be compressed to run on commodity and edge hardware with modest accuracy loss (Sections 4–6); and a mature ecosystem now exists to deploy them (Section 6.3). This section argues that these facts, taken together with how agentic systems are actually built, make SLMs the *default* execution substrate for AI agents, with large models reserved for a shrinking set of genuinely hard calls. The argument follows and extends the position of Belcak, Heinrich, Diao, et al. [20]; we situate it within the broader literature on LLM–SLM collaboration [21, 22] and connect it to the deployment evidence consolidated in this paper.

7.1 What Agents Delegate to Language Models

An AI agent is a software system in which one or more language models operate under a control loop that also includes tools, memory, and a scaffold of prompts and parsers [20]. The language model in such a system is rarely asked to hold an open-ended conversation. It is asked to perform bounded sub-tasks: decide which tool to call and with what arguments, extract fields from a document, classify an intent, rewrite a query, summarize a retrieved passage, produce a short code patch, or check whether a step succeeded. These calls share three properties. They are *narrow*—each exercises a small slice of the model's competence. They are *repetitive*—the

same handful of call types recur thousands of times in production. And they are *format-constrained*—the output must parse as JSON, match a function signature, or fall within a fixed label set, or the surrounding code breaks [20].

None of these properties rewards the breadth of a frontier LLM. A model that can also write a sonnet, debate philosophy, and reason across a 200-page contract is carrying capacity that the agent never invokes on the call in question. Belcak, Heinrich, Diao, et al. [20] formalize this as the observation that agents use only a narrow subset of language-model functionality, and that agentic systems demand strict behavioral alignment to code interfaces—exactly the setting in which a specialized small model can match or exceed a general large one.

7.2 The Case for SLMs in Agents

7.2.1 Capability Sufficiency

The first premise is that SLMs are already good enough for delegated sub-tasks. On instruction following, tool selection, and structured extraction—the operations that dominate agent traces—sub-10B models trained or tuned for the purpose are competitive with frontier models. Salesforce's xLAM family [69] illustrates the point: its 7B function-calling model ranked at the top of the Berkeley Function-Calling Leaderboard, and its 1B variant was reported to surpass GPT-3.5-Turbo and Claude-3 Haiku on the same task despite being one to two orders of magnitude smaller. Unified-reasoning SLMs extend this to multi-step tasks: Qwen3 [17] exposes a controllable "thinking budget" so that an agent can spend inference compute only on the calls that need it, and distilled reasoning models such as Phi-4-mini-reasoning [86] and the DeepSeek-R1 distillations [85] bring competition-mathematics performance into the 3–8B range. Belcak, Heinrich, Diao, et al. [20] argue that objections based on scaling laws (AV1: larger is permanently better) understate SLM capability because scaling-law studies hold architecture fixed, exclude task-specific fine-tuning, and predate inference-time reasoning—all three of which disproportionately help small models.

7.2.2 Economics

The second premise is cost. Belcak, Heinrich, Diao, et al. [20] estimate that serving a 7B SLM is 10–30× cheaper than serving a 70–175B LLM once latency, energy, and FLOPs are accounted for, and that parameter-efficient fine-tuning lets a small model be specialized for a new agent role in GPU-hours rather than GPU-weeks. The deployment figures consolidated in this paper are consistent with that range: Table 12 shows a 5–20× cost advantage and comparable or better latency for a quantized 3B model relative to a 70B model on chatbot and code-completion workloads, and an absolute advantage in offline settings where the large model cannot run at all. Because an agent amortizes model cost over many calls per user task, a constant-factor saving on each call compounds into the difference between a viable and an unviable product margin.

7.2.3 Behavioral Alignment and Operational Fit

The third premise is control. An agent needs its model to emit exactly the format the scaffold expects, and to do so reliably across a long tail of inputs. Constraining a small specialized model to a narrow output distribution is easier than suppressing the much larger behavioral range of a frontier model, and fine-tuning a small model on the agent's own logged traces closes the residual gap [20]. Belcak, Heinrich, Diao, et al. [20] note a virtuous cycle here (their argument A7): agent interactions generate high-quality, in-distribution training data essentially for free, which is precisely what specialization of a small model requires. Operationally, small models are also easier to version, roll back, run on-premises for data-residency reasons, and co-locate with the tools they call.

7.3 Heterogeneous SLM–LLM Systems

The position is not that LLMs disappear from agents but that the default inverts: SLMs handle the repetitive language errands, and a strong generalist model is invoked only for planning, open-ended decomposition, or calls that genuinely need long context or broad world knowledge [20]. Figure 8 sketches the arrangement. Realizing it requires a mechanism to decide, per call, which model to use. Chen, Zhao, and Han [22] organize the collaboration literature into five interaction patterns—pipeline, routing, auxiliary, distillation, and fusion—of which routing is the most direct fit for agents.

7.3.1 Learned Routing

RouteLLM [67] trains a lightweight router on human-preference data to predict, for each query, whether a weak model's answer would be judged as good as a strong model's, and dispatches accordingly; the authors report cost reductions of more than 2× on standard benchmarks with no measurable quality loss, and transfer of the trained router to unseen strong/weak model pairs. Division-of-Thoughts [68] pushes routing below the query level, decomposing a task into sub-questions and assigning each to a local SLM or a cloud LLM according to

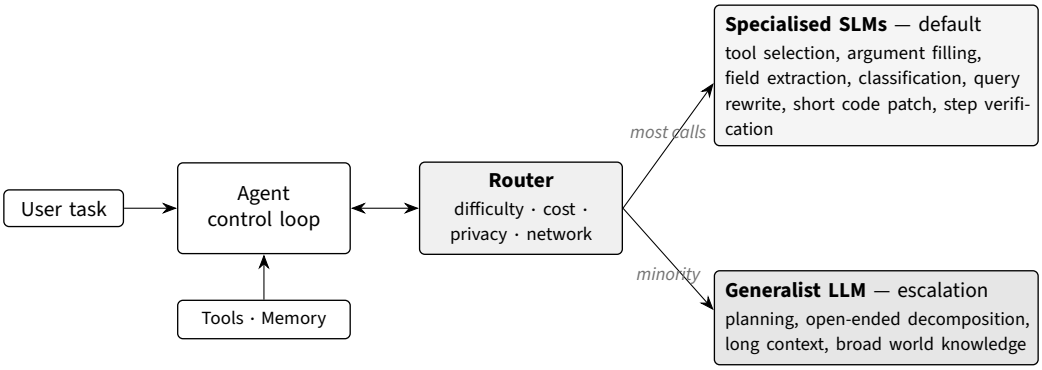


Figure 8. Heterogeneous SLM–LLM agent. A per-call router sends the majority of delegated sub-tasks to specialised small models and escalates only planning-like or long-context calls to a generalist LLM; tools and memory are shared. After Belcak, Heinrich, Diao, et al. [20].

estimated difficulty, for on-device agents. The open problems are difficulty estimation without an oracle, routing objectives that jointly weigh accuracy, latency, privacy sensitivity, and network availability, and handoff that does not produce a visible seam in output quality [130].

7.3.2 Production Instances

Two large deployments already embody the heterogeneous pattern. Apple's Foundation Models framework pairs an on-device model of roughly 3B parameters, quantized with 2-bit quantization-aware training and KV-cache sharing, with a Private Cloud Compute server model built on a Parallel-Track Mixture-of-Experts transformer, and routes to the server only when the on-device model is expected to fall short [19]. The on-device model is reported to be competitive with Qwen3-4B and Gemma-3-4B on its target tasks. Third-party providers can now register cloud models behind the same API surface as the local model, making the SLM the default and the LLM an opt-in escalation [126]. Samsung's Gauss and Google's Gemini Nano occupy the same design point on their respective platforms [127].

7.4 Converting an LLM Agent to an SLM Agent

For teams with an existing LLM-only agent, Belcak, Heinrich, Diao, et al. [20] propose a six-step conversion pipeline that we summarize here because it operationalizes the position. **S1, instrumentation:** log every non-interface model call (inputs, outputs, latency) from the running agent, with encryption and access control. **S2, curation:** strip personally identifiable and protected health information and application secrets from the logs. **S3, clustering:** apply unsupervised clustering to the curated calls to surface the recurring request types—these are the specialization targets. **S4, selection:** for each cluster, choose a candidate SLM by matching required capability against benchmarks, license, and deployment constraints. **S5, specialization:** fine-tune the chosen SLM on the cluster's curated data, typically with LoRA or QLoRA. **S6, iteration:** re-run the loop periodically as the traffic distribution drifts. The pipeline is attractive precisely because the data it needs is a by-product of running the agent, and because each cluster converted removes a recurring frontier-model bill.

7.5 Barriers to Adoption

Belcak, Heinrich, Diao, et al. [20] identify three reasons the shift has lagged the technical case. First, large centralized-inference investments create sunk-cost inertia and an operational bias toward one big model behind one API (B1). Second, SLMs are still evaluated on generalist leaderboards (MMLU, broad chat benchmarks) that are misaligned with agentic utility, so their fitness for delegated sub-tasks is under-measured (B2)—an evaluation gap this paper's consolidated tables also reflect, since public reporting concentrates on the same generalist benchmarks. Third, SLM releases receive far less attention than frontier launches, so awareness lags capability (B3). None of these is a technical obstacle, which is the point: the barriers are those of infrastructure, measurement, and attention, and each is already eroding as routing frameworks mature, agent-specific benchmarks such as the Berkeley Function-Calling Leaderboard gain traction, and on-device platforms ship SLMs to billions of devices by default.

8 Domain-Specific Applications

The domains below are where SLM deployment is most advanced, and each is an agentic setting in the sense of Section 7: a workflow of bounded, repeated language operations—extract a diagnosis code, classify a clause,

complete a function, translate a passage—executed under regulatory or cost constraints that make a cloud frontier model impractical. Domain fine-tuning of a small model is the LLM-to-SLM conversion pipeline (Section 7.4) applied to a vertical.

8.1 Healthcare and Clinical NLP

Healthcare NLP presents unique requirements: high factual accuracy, interpretability, HIPAA/GDPR compliance, and compatibility with clinical workflows. Large general-purpose LLMs are increasingly restricted from processing identifiable patient data due to data residency regulations, creating a strong pull toward on-premises SLMs.

The Meerkat family [90] demonstrates the potential of domain-specific SLM fine-tuning. Built on a 7B base with instruction fine-tuning on curated clinical QA datasets and chain-of-thought reasoning traces from medical board examinations, Meerkat achieves 85.7% on USMLE Step 1—surpassing GPT-4 (86.0% in zero-shot) while being deployable on a single A100 GPU rather than a multi-GPU cloud cluster. Meerkat's clinical reasoning is particularly strong on diagnostic pattern matching, outperforming GPT-4 by 4% on complex differential diagnosis tasks [90].

OptimCLM [91] applies the full compression pipeline—knowledge distillation, structured pruning, and 4-bit quantization—to clinical language models, reducing a 7B model to under 1B while preserving 95% of its accuracy on ICD coding and discharge summary generation tasks. This makes clinical NLP feasible on hospital workstations without GPU infrastructure.

The broader SLM healthcare ecosystem includes BioMedLM (PubMed-trained 2.7B model), clinical note generation models, and radiology-specific models. A key trend is the use of retrieval-augmented generation (RAG) [131] to anchor SLM outputs to verified clinical knowledge bases, reducing hallucination rates in patient-facing applications.

8.2 Legal NLP

Legal document processing—contract review, compliance monitoring, regulatory change analysis, case law search—involves long, precisely structured documents where hallucination is particularly dangerous. SLMs address legal NLP deployment constraints in two ways: (1) by enabling on-premises deployment to satisfy attorney-client privilege and data security requirements; and (2) by enabling cost-effective processing of the massive volumes of routine legal work that would be prohibitively expensive with frontier LLMs.

Pilot deployments of domain-adapted SLMs (3B–7B range, fine-tuned on contract corpora) have demonstrated 30–40% reductions in administrative processing time for standard contract review workflows. Models fine-tuned on specific regulatory domains (GDPR, SOC 2, FINRA) consistently outperform general-purpose GPT-4 on domain-specific compliance classification tasks, owing to the depth of in-domain knowledge concentration.

8.3 Code Generation

Code-focused SLMs have matured into a category distinct from general SLMs, with specialized pre-training corpora (GitHub, Stack Overflow, documentation), code-aware tokenization, and evaluation on function-level benchmarks (HumanEval, MBPP, SWE-Bench).

Code LLaMA [92]
: Meta's family of code models fine-tuned on LLaMA-2 (7B, 13B, 34B). Code LLaMA 7B achieves 33.5% HumanEval pass@1.

StarCoder2 [93]
: BigCode's 3B/7B/15B models trained on The Stack v2 (600+ programming languages). StarCoder2-7B achieves 46.3% HumanEval pass@1.

DeepSeek-Coder [94]
: A 1B–33B family trained from scratch on 2T code tokens. The 6.7B variant achieves 47.6% HumanEval pass@1, outperforming GPT-3.5 on code completion.

Codestral [132]
: Mistral's 22B code model trained on 80+ programming languages with fill-in-the-middle (FIM) training for IDE integration.

8.4 Education and Multilingual NLP

Multilingual SLMs address the linguistic coverage gap in AI assistants: the majority of LLM pre-training data is English, leaving speakers of lower-resource languages with significantly degraded performance. Aya 8B [95] is fine-tuned on 204 million multilingual instructions spanning 101 languages, with strong performance across African, South Asian, and Southeast Asian languages that are severely underrepresented in standard benchmarks.

Qwen2.5 7B [14] is pre-trained on 18T tokens with strong multilingual coverage including Chinese, Japanese, Korean, Arabic, and multiple European languages, serving as the de facto open-weight SLM for multilingual enterprise deployments. Gemma 3's multilingual extension [39] adds instruction-tuning data for 40+ languages at the 4B and 12B scales.

Educational applications include on-device tutoring systems (leveraging Phi-3 or SmolLM on tablets), interactive language learning assistants with real-time feedback, and low-connectivity educational tools for rural and developing-world deployments. The key requirement for educational SLMs is not just multilingual capability but also low hallucination rates and age-appropriate safety filtering.

9 Discussion

9.1 What the Consolidated Evidence Shows

Read together, the taxonomy (Section 3), the compression review (Section 4), and the consolidated reported performance (Section 5) support four claims.

Capability per parameter has risen sharply, and unevenly. The best reported 3–4B models now match the 2023 state of the art for 13B-class systems on mathematics, code, and knowledge benchmarks. The dominant explanatory variable in the reported record is not architecture but training data: models of near-identical size trained on different corpora differ by 15–25 points on knowledge tasks, a gap no architectural choice at this scale approaches. The practical implication for model builders is that data curation and synthetic-data pipelines dominate architectural novelty as a lever on SLM quality.

Compression is a first-class capability, not an afterthought. INT8 is reported as effectively lossless and INT4 as costing 1–4 points, with smaller models and mathematical reasoning at the higher end. Because autoregressive inference is memory-bandwidth-bound, 4-bit quantization is simultaneously a storage optimization and a near-linear speedup on edge hardware. Quantization consistently outperforms pruning below 4B parameters [26], which simplifies the practitioner's decision: quantize aggressively, prune only when a structured size reduction is independently required.

Right-sizing beats scaling for bounded tasks. A quantized 3.8B model that clears a task's accuracy threshold, running on hardware that already exists in the user's hand, is preferable on every axis—cost, latency, energy, privacy—to a larger model that also clears it. The reported numbers make this concrete: a 3–4B model at INT4 occupies roughly 2 GB, runs at interactive speed on a phone or a Jetson, and costs a small fraction of a 70B model per token. The scaling-era assumption that a larger model is always the safer default is inverted for any task whose requirements can be stated.

The evidence base itself is the weak point. Every figure in Section 5 is a reported value drawn from heterogeneous sources with different harnesses, prompt formats, and self-reporting incentives. The consolidated tables are internally reconciled and provenance-tagged (Appendix A), but they are not a substitute for a controlled re-evaluation. Section 10.7 specifies the verification studies that would convert the load-bearing claims from "reported" to "measured."

9.2 Implications for Agentic Systems

The agentic argument of Section 7 rests on these claims rather than restating them. Capability-per-parameter growth is what makes an SLM *sufficient* for a delegated sub-task; compression and edge deployability are what make it *cheap* and *private*; right-sizing is the design discipline that turns a heterogeneous SLM–LLM system from an option into the default. Two cross-cutting observations sharpen the picture. First, routing must be capability-aware, not size-aware: the reported multilingual and safety gaps (Section 5.7) mean that a naive "small model unless hard" router will silently degrade non-English and safety-sensitive calls. Second, the specialization threshold near 3B (Section 5.3) implies that the economical unit in an agent is often a *fine-tuned* sub-3B model per recurring call type, not a single general SLM—exactly the target of the LLM-to-SLM conversion pipeline.

9.3 Democratization and Sustainability

Deploying a 3B model that answers grade-school mathematics correctly on an $80 single-board computer is a qualitative change in access. Organizations facing cloud-API cost, data-sovereignty rules, or unreliable connectivity can now run capable NLP locally with no per-query cost and no data egress. The environmental case is directional rather than precise: reported per-inference energy for a small model on modest hardware is one to two orders of magnitude below a frontier-model query in a datacenter, and if even a modest fraction of today's classification, extraction, and short-generation traffic were served by right-sized SLMs, the aggregate saving would be large. These figures are estimates and should be treated as such.

9.4 Limitations

Beyond the evidence-base limitations already stated (Section 5.1.5): the review is English-centric in its task coverage; the ecosystem moves faster than any static snapshot, so models released after the collection window are discussed only qualitatively; the taxonomy's boundaries (the 10B threshold, the treatment of MoE by active parameters) are pragmatic conventions, not natural kinds; and the agentic argument, while grounded in a production-scale position paper [20] and two large deployments, is a projection about where the field is heading, not a measured outcome.

9.5 Ethical Considerations

Safety alignment is proportionally harder at small scale: several research-grade SLMs lack the safety training of their larger siblings, and fine-tuning can erode what alignment exists [133]. Practitioners deploying SLMs—and especially those wiring them into agents with tool access—should treat application-level safety filtering and output monitoring as mandatory, not optional. Compression carries its own risk: pruning and quantization can degrade performance unevenly across dialects, languages, and demographic groups [134], and the reported advantage of quantization over pruning is largest precisely on multilingual tasks [26], which suggests pruning disproportionately harms non-English users. Fairness-aware compression objectives and per-group performance floors for high-stakes applications are open requirements. Against these risks, on-device SLMs advance privacy (no data leaves the device), accessibility (no connectivity required), and cost (no subscription), which matters most for the healthcare, education, and agricultural applications of Section 8 in under-resourced settings.

10 Challenges and Future Directions

Despite remarkable progress, SLMs face persistent challenges that define the frontier of the field, and the agentic framing of Section 7 adds a set of its own. We organize these into research clusters, closing with the empirical-verification agenda for this paper's consolidated evidence base.

10.1 Hallucination Mitigation in Small Models

Hallucination—confident generation of factually incorrect content—is an intrinsic failure mode of autoregressive language models, and it is more pronounced in smaller models [135]. The core issue is information capacity: a 3B model's parameters can encode far fewer facts than a 175B model, and when queried about knowledge at the boundary of its training distribution, small models are more likely to confabulate plausible-sounding but incorrect responses.

Mitigation strategies include: (1) *RAG* [131], which anchors SLM generation to retrieved documents rather than parametric memory; (2) *uncertainty quantification* through sampling and ensemble methods; (3) *RLHF with truthfulness rewards* to penalize confident incorrect responses; and (4) *calibration post-processing* to align model confidence to actual accuracy.

10.2 Reasoning Limitations Under Parameter Constraints

Complex multi-step reasoning—particularly in mathematics, formal logic, and scientific inference—improves roughly as a power law with parameter count [6]. At the 1B–7B scale, SLMs struggle with problems that require holding many intermediate states simultaneously or with deep deductive chains.

Emerging directions include: (1) *Test-time scaling*: allocating additional inference compute (more generation tokens, repeated sampling, tree-of-thought search) to substitute for model capacity; DeepSeek-R1 [85] achieves frontier-class reasoning in its 7B distilled variant through extended chain-of-thought generation. (2) *Neuro-symbolic integration*: augmenting SLMs with symbolic solvers (SAT solvers, theorem provers, calculators) to handle formal reasoning externally. (3) *Speculative decoding* [136] using a small draft SLM to propose tokens verified by a larger model, enabling faster inference from larger models when latency permits.

10.3 Open Problems for Agentic SLM Systems

Section 7 argues that SLMs are the default substrate for agents; several problems stand between that position and routine practice. **Difficulty estimation without an oracle:** a router must decide, cheaply and before generation, whether a given call exceeds the local model's competence; current approaches use learned win-prediction [67] or sub-task decomposition [68], but calibration under distribution shift is unsolved. **Multi-objective routing:** the decision must jointly weigh accuracy, latency, energy, monetary cost, privacy sensitivity, and network availability, and the reported multilingual and safety gaps (Section 5.7) mean a size-only router silently degrades non-English and safety-critical calls. **Seamless handoff:** escalation from SLM to LLM mid-task must not produce a visible

discontinuity in style or quality. **Agent-appropriate evaluation:** generalist leaderboards do not measure the tool-selection, argument-filling, and format-adherence skills agents actually use [20]; function-calling leaderboards [69] are a start but cover a narrow slice. **Fleet management:** an agent built from many fine-tuned sub-3B specialists (Section 7.4) raises questions of versioning, drift detection, and retraining cadence that single-model deployments do not face. Apple's privacy-preserving cloud escalation [19, 125] shows the pattern is production-viable; generalizing it beyond a single vendor's stack is open.

10.4 Hardware Co-Design and NPU Optimization

The efficiency of SLM inference depends as much on hardware as on model architecture. Modern consumer hardware increasingly includes dedicated AI accelerators: Apple ANE, Qualcomm Hexagon NPU [137], Samsung Exynos NPU, and Intel NPU. These accelerators are optimized for specific operation types (INT4/INT8 matrix multiplication) and data layouts.

A critical open problem is *hardware-model co-design*: developing SLM architectures whose attention patterns, FFN widths, and layer counts align with the data bus widths and tile sizes of target NPUs. GQA head counts of 8 or 16 align well with SIMD widths on ARM Cortex-A processors, while head counts of 32 may be suboptimal. Future SLMs trained with hardware constraints as first-class optimization objectives—rather than retrofitting quantization post-hoc—will likely yield significant efficiency improvements.

10.5 Ethical Considerations: Bias Amplification in Compressed Models

Model compression introduces an underexplored ethical risk: *bias amplification*. Pruning and quantization may disproportionately affect model performance on underrepresented demographic groups, dialects, and languages, as the model's remaining capacity may preferentially serve the majority distributions present in training data [134]. Kurz and Zhao [26] observe that quantization's advantage over pruning is most pronounced for multilingual tasks, suggesting that pruning disproportionately degrades non-English performance.

Research priorities include: (1) *fairness-aware compression* that includes demographic parity metrics as part of the compression objective; (2) *representation auditing* of compressed models across language, dialect, gender, and race; and (3) *minimum performance floors* for safety-critical applications, ensuring compressed models meet baseline accuracy thresholds for all user populations.

10.6 Sustainable AI: SLMs as a Green AI Alternative

The energy consumption of training and serving frontier LLMs has become a significant environmental and economic concern [138, 139]. Training GPT-3 emitted an estimated 552 metric tons of CO_2 equivalent; training a comparable SLM from scratch consumes orders of magnitude less energy, and inference costs are proportionally lower.

The "SLMs as green AI" framing extends beyond training to the lifecycle of deployed models. A deployment of 10,000 concurrent SLM instances on consumer hardware consumes less total energy than a single multi-GPU LLM server, while the carbon amortization benefits of on-device inference (utilizing hardware already present in user devices) are substantial. As AI regulation increasingly incorporates carbon disclosure requirements, the energy efficiency profile of SLMs will become a competitive differentiator.

10.7 Empirical-Verification Opportunities

Because this review synthesizes reported performance rather than producing original benchmark runs, the most immediately valuable follow-up work is targeted empirical verification of the reported claims that do the most work in practitioner decisions. Four empirical studies would disproportionately reduce the residual uncertainty in the evidence base reviewed here:

Harness-consistent reruns of the headline SLMs on the headline benchmarks
: A controlled re-run of the seven most-cited SLMs (Phi-4-mini, Phi-4, Qwen2.5-3B, Qwen2.5-7B, Gemma-3-4B, LLaMA-3.2-3B, Mistral-7B) on MMLU, GSM8K, HumanEval, and MT-Bench under a single `lm-evaluation-harness` configuration would anchor every vendor-reported figure to an independent reproduction.

Quantization-degradation measurement
: A controlled comparison of FP16 vs. INT8 vs. INT4 accuracy across the seven headline SLMs and four benchmarks, using the GGUF, GPTQ, and AWQ toolchains referenced in the vendor reports, would sharpen the 1–4-point degradation figure reported in the literature.

Edge-throughput reproduction
: Timed runs of the same seven SLMs in their best-supported INT4 format on a Raspberry Pi 5, Jetson Orin

Nano, Snapdragon reference device, and recent laptop CPU would calibrate the reported throughput ranges against modern silicon.

QLoRA-gain reproduction
A controlled QLoRA (rank 64, α = 128) adaptation of the seven SLMs on SST-2, MNLI, ARC-C, GSM8K, and SQuAD 2.0 would quantify the reported fine-tuning gains under a single adaptation recipe.

These four studies are practical on a single modern GPU on the time scale of weeks and would convert several of the consolidated claims in this review from "reported by sources" to "empirically verified." They do not replace the review; they strengthen the claims that practitioners depend on most.

10.8 Open Research Directions Beyond Empirical Verification

Beyond direct empirical verification, several promising research directions emerge from this review:

Multilingual SLM evaluation
Extending the consolidated performance evidence to non-English benchmarks would reveal whether the English-centric performance hierarchy reported in the literature holds across languages, particularly for models with multilingual training data (Qwen2.5, Gemma 3, BLOOM).

On-device continual learning
Developing methods for incremental adaptation of deployed SLMs to new data without full retraining would enable personalization and domain adaptation at the edge.

SLM ensembles vs. single LLM
Investigating whether ensembles of specialized SLMs (one for math, one for code, one for general knowledge) with a lightweight router can match single-LLM performance at lower aggregate cost.

Multimodal SLMs
Evaluating vision-language SLMs such as Phi-3-vision, Gemma 3 with vision, and Gemma 3n [89] on multimodal benchmarks would extend the comparison to the rapidly growing multimodal inference domain.

Hybrid architectures
The emergence of non-transformer SLM architectures (SambaY [140], Gated Delta Networks [141]) suggests that the transformer's dominance may not extend to the edge, where linear-time inference is particularly valuable.

Federated fine-tuning
Developing privacy-preserving fine-tuning protocols that aggregate gradient updates across edge devices would enable collaborative model improvement without centralized data collection.

11 Conclusion

This paper set out to do two things at once: survey the small-language-model landscape as it stands in 2026, and argue a position about where it is going. On the first, we cataloged 123 documented SLM variants under a four-dimensional taxonomy, tracked the 2025–2026 shift toward hybrid state-space architectures, unified reasoning models, and mobile-first multimodal families, reviewed the compression techniques specific to the sub-10B regime—where quantization consistently outperforms pruning [26]—and consolidated publicly reported performance for a 17-model cohort across 15 tasks, six hardware tiers, and three precisions, with every figure traced to a dated source rather than an original run.

On the second: the "less is more" thesis is no longer only about efficiency. The consumer of language intelligence that is growing fastest is the AI agent, and the agent does not want a generalist. It issues many narrow, repetitive, format-constrained calls, and for those calls a small specialized model is sufficiently capable, 10–30× cheaper to serve, cheaper to adapt, and easier to hold to a code interface [20]. The consolidated evidence in this paper supports each premise of that argument, and two production-scale deployments—Apple's on-device/server split and the routing-based systems surveyed in Section 7—already embody it. The barriers that remain are those of sunk infrastructure, benchmark mis-measurement, and attention, not of capability.

The work ahead is concrete. Reasoning still shows a performance cliff below roughly 7B parameters that inference-time compute only partly closes; hallucination is worse at small scale; compression can amplify bias; hardware and model architecture are still designed separately; and the consolidated evidence base needs controlled verification of the handful of claims that practitioners most depend on (Section 10). None of these is a reason to wait. The future of applied NLP is not one frontier model answering every query through a cloud API. It is a fabric of small, specialized, mostly on-device models doing the ordinary work, calling a large model only when the task genuinely demands it. In that architecture, less is not a limitation. It is the design.

Declarations

Funding

[No funding was received for this work.]

Competing interests

The authors declare that they have no competing interests.

Ethics approval and consent to participate

Not applicable. This study is a literature review and analysis of publicly available model documentation; it involves no human participants, human data, or animals.

Consent for publication

Not applicable.

Data availability

This paper is a review. All performance figures are *reported* values compiled from public sources—vendor technical reports, official model cards, community leaderboards, and third-party evaluations—and are not the product of original benchmark runs. Every consolidated figure is traced to a dated, citable source in Appendix A. The model-ecosystem inventory analysed in Section 3 and Appendix B is generated from the dataset `merged_llm_database.json` (snapshot September 2026), provided with the manuscript source files.

Code availability

The scripts that generate the ecosystem inventory tables and the figures (`generate_model_tables.py`, `generate_figures_v2.py`) are provided with the manuscript source files. No model-training or model-evaluation code was used to produce the results in this paper, which are compiled from the published literature.

Author contributions

M.G.R.: Conceptualization, Methodology, Investigation, Data curation, Software, Writing — original draft, Visualization. **H.A.:** Conceptualization, Methodology, Writing — review & editing, Supervision, Project administration. **S.J.A.:** Investigation, Validation, Writing — review & editing. **A.A.:** Validation, Writing — review & editing, Funding acquisition. All authors read and approved the final manuscript.

Acknowledgements

[The authors thank . . . .]

A Data Provenance and Supplementary Protocols

This appendix supports the reproducibility of the *review itself*: it documents the sources of the reported figures consolidated in Sections 5–6, and provides the reference prompt templates, quantization-pipeline specifications, and QLoRA hyperparameter settings under which those figures were reported. The appendix does not report original experimental runs.

A.1 Per-Figure Provenance Table

To be populated at submission: a per-figure provenance table listing, for every reported performance value appearing in Sections 5–6, the following fields: (a) model (name + Hugging Face commit hash or release version); (b) benchmark and metric variant; (c) quantization precision and toolchain; (d) hardware configuration; (e) source citation (vendor technical report, model card, leaderboard entry, or third-party study) with a stable identifier (DOI, arXiv ID, or archival URL); (f) extraction date; and (g) any conflict-resolution decision applied under the rules of Section 5.1.4.

The remaining subsections of this appendix document the reference conditions—prompt formats, quantization toolchains, adaptation hyperparameters—that the reviewed sources most commonly report; these are preserved here so that readers can interpret any individual reported figure against the conditions it was measured under.

A.2 Prompt Formats

The reviewed source materials commonly preserve each model's own instruction wrapper while following a shared task-payload structure, which is summarized here in prose. **Multiple-choice question answering** presents the question followed by the enumerated options (A–D) and an "Answer:" cue, with the model expected to emit a single

option letter. **Mathematical reasoning** prepends an instruction to solve the problem step by step and appends a "Let's think step by step" cue, so that a chain-of-thought precedes the final numeric answer. **Extractive question answering** supplies the context passage, then the question, then an instruction to answer from the context and to respond "unanswerable" when the context does not contain the answer. **Abstractive summarization** supplies the source article preceded by an instruction to produce a two- to three-sentence summary. Where a source reports few-shot results, the same fixed set of exemplars is prepended to the payload for every model on a given task. Because sources differ in the exact wording of these wrappers and in answer-extraction heuristics, the consolidated figures are compared only within tasks and, wherever possible, within a single reporting source.

A.3 Quantization and Compression Pipeline

To complement the methodology section, the compression workflows reported across the reviewed sources are summarized here. The reviewed literature reports five precision or compression settings across the consolidated comparison:

FP16 baseline
: Standard half-precision inference used as the reference condition.

GGUF Q4_K_M
: Mixed-precision 4-bit quantization via `llama.cpp`, assigning higher precision to the most sensitive tensors.

AWQ 4-bit
: Activation-aware weight quantization with group size 128 and calibration on held-out text sequences.

GPTQ 4-bit
: Layer-wise post-training quantization using second-order curvature approximations.

bitsandbytes NF4
: 4-bit NormalFloat quantization with double quantization, matching the QLoRA training regime.

For compression stress tests, the sources report SparseGPT [47]-induced 50% unstructured sparsity applied before 4-bit quantization. Reported figures for this combined pruning-plus-quantization setting are retained in the provenance tables for sensitivity analysis rather than the main comparative tables, because support for sparse inference remains hardware dependent and is not uniformly reported across the reviewed sources.

A.4 Edge Measurement Protocols Reported in the Source Literature

The deployment figures consolidated in Sections 5–6 are drawn from source materials whose stated measurement protocols typically include a multi-minute warm-up to stabilize thermal and frequency behavior before measurement. Across the reviewed edge-deployment sources, the reported per-model, per-device measurements cover:

Time-to-first-token (TTFT)
: Reported latency from prompt submission to first generated token, typically averaged over 50 prompts with 128-token inputs.

Sustained throughput
: Reported mean tokens-per-second over 256-token generations.

Peak memory
: Reported maximum RAM or unified-memory consumption during generation.

Load time
: Reported elapsed time from process start until the model is ready to serve requests.

Energy per inference
: Reported device-level power estimates where the source provides counter- or telemetry-based measurements.

Because these reporting conventions differ in small but potentially consequential ways across sources (warm-up duration, prompt-length distribution, power-counter granularity), readers should treat the consolidated figures as indicative rather than precisely comparable; the conflict-resolution rules in Section 5.1.4 are applied whenever discrepant protocols yield materially different reported values for the same tuple.

A.5 Hyperparameter Search Space

The right-hand column records the QLoRA configuration that the reviewed sources most commonly report; where a source used a different rank, target-module set, or schedule, the deviation is noted in the per-figure provenance table so that its reported gain is not compared directly against gains obtained under the common recipe.

Table 13. QLoRA hyperparameter configuration most commonly reported by sources.

Hyperparameter	Search Space	Selected Value
LoRA rank r	{16, 32, 64, 128}	64
LoRA scaling α	{16, 32, 64, 128, 256}	128
LoRA dropout	{0.0, 0.05, 0.1}	0.05
Learning rate	$\{10^{-5}, 5 \times 10^{-5}, 10^{-4}, 2 \times 10^{-4}, 5 \times 10^{-4}\}$	2×10^{-4}
Effective batch size	{8, 16, 32}	16
Max epochs	{3, 5, 10}	5
Warmup steps	{50, 100, 200}	100
Target modules	{q,v}, {q,k,v,o}, {all linear}	{q,k,v,o}
Weight decay	{0.0, 0.01, 0.1}	0.01
Scheduler	{cosine, linear}	cosine

A.6 Extended Results Tables

Table 14. Representative five-shot results for selected models.

Model	ARC-C	HellaSwag	GSM8K	MMLU	SST-2	MNLI
Phi-4-mini-Inst.	85.2	71.4	89.3	69.8	94.1	75.8
Qwen2.5-7B-Inst.	91.2	81.5	89.5	74.2	95.2	77.3
LLaMA 3.2-3B-Inst.	78.8	79.1	77.4	64.5	91.5	68.2
SmolLM2-1.7B-Inst.	63.1	63.5	44.8	53.4	86.4	59.8

Table 15. Extended zero-shot results across a broader task set (FP16, instruction-tuned variants). All values are reported figures.

Model	ARC-E	ARC-C	HellaSwag	TriviaQA	SQuAD 2.0	GSM8K	MATH	HumanEval	MBPP	MMLU	SST-2	MNLI
Phi-4-mini-Inst.	92.1	83.7	69.1	68.5	74.2	88.6	64.0	72.0	74.8	67.3	92.4	72.1
Qwen2.5-7B-Inst.	94.8	90.1	80.0	72.4	76.8	88.7	60.4	84.8	79.2	72.6	93.8	74.6
Gemma 2-9B-Inst.	95.2	89.8	87.1	71.8	75.5	84.9	51.3	68.9	73.5	71.3	93.5	73.8
Gemma 3-4B-Inst.	92.4	85.2	78.4	64.2	70.8	82.1	53.7	64.6	69.3	65.8	91.2	70.4
LLaMA 3.2-3B-Inst.	88.5	76.1	77.2	58.6	65.4	75.6	46.7	58.5	62.4	61.8	88.2	63.5
LLaMA 3.2-1B-Inst.	78.2	62.3	63.8	42.5	52.8	47.2	24.1	35.4	40.2	48.6	82.5	54.2
Mistral 7B-Inst. v0.3	91.8	82.7	73.5	65.8	68.4	81.9	41.6	66.5	68.3	63.0	90.8	68.2
Ministral 8B-Inst.	93.2	85.4	76.8	67.2	71.2	83.5	45.2	69.5	71.8	65.7	92.1	71.5
Qwen2.5-3B-Inst.	90.2	78.6	72.1	58.8	66.5	79.8	50.2	65.2	67.8	65.6	89.5	66.8
Qwen2.5-0.5B-Inst.	68.4	52.1	48.3	28.5	38.2	38.5	18.7	22.0	28.4	47.5	78.2	48.5
SmolLM2-1.7B-Inst.	74.5	59.8	60.2	35.4	52.3	42.6	19.3	28.7	35.6	50.1	82.6	54.2
SmolLM2-360M-Inst.	55.8	38.2	42.5	18.2	32.4	15.8	5.2	8.5	12.3	33.4	72.4	42.8
TinyLlama 1.1B	62.4	47.5	53.8	28.8	42.5	18.4	6.8	12.2	16.8	36.2	76.5	46.2
StableLM 2 1.6B	72.8	55.2	57.6	32.5	48.6	32.1	12.4	22.5	28.9	44.3	80.2	52.5

A.7 Detailed Hardware Specifications

Table 16. Detailed hardware specifications for edge deployment experiments.

Specification	Raspberry Pi 5	Jetson Orin Nano	Galaxy S24+	Laptop
CPU	BCM2712 4×A76 @ 2.4GHz	6×A78AE @ 1.5GHz	Snapdragon 8 Gen 3	Ryzen 7 7840HS
GPU	VideoCore VII	1024 CUDA (Ampere)	Adreno 750	Radeon 780M
NPU	None	None	Hexagon	Ryzen AI
RAM	8 GB LPDDR4X	8 GB LPDDR5 (unified)	12 GB LPDDR5X	32 GB DDR5-5600
Memory bandwidth	25.6 GB/s	68 GB/s	51.2 GB/s	89.6 GB/s
Storage	microSD / NVMe	NVMe	UFS 4.0	NVMe SSD
TDP	12W	25W	~8W sustained	35–54W
Inference engine	llama.cpp	MLC LLM	ExecuTorch	llama.cpp

A.8 Human-Evaluation Conventions in the Source Literature

For subjective tasks such as summarization quality and instruction following, the reviewed sources that report human judgments typically rate outputs along three axes—**relevance**, **coherence**, and **factual accuracy**—on a 1–5 Likert scale, with model identity hidden from annotators and inter-annotator reliability reported via a Krippendorff-style coefficient. MT-Bench-style results substitute a strong LLM judge for human raters. Because these protocols differ across sources in rubric wording and judge model, the consolidated instruction-following column (MT-Bench) should be read as indicative, and human-rated summarization scores are reported only as ranges.

B Broader Small-Model Ecosystem Context

The standalone inventory drafts in this project tracked the wider model ecosystem surrounding the evaluated SLM cohort. Their main function was to contextualize the paper within the rapidly changing 2025–2026 release cycle, where compact open models, open-weight reasoning models, and frontier API systems evolved in parallel. The distilled summary below preserves the most relevant ecosystem information inside the main paper.

B.1 Landscape Snapshot

Three ecosystem trends are especially relevant to the present study:

Capability compression
: Reasoning, long-context handling, and multimodality have moved into the 1B–8B range, narrowing the gap between deployable SLMs and cloud-scale LLMs.

Open-weight acceleration
: Families such as Qwen, Phi, Gemma, SmolLM, DeepSeek distills, Granite, and MiniCPM have expanded the design space for self-hosted deployment (Figure 11).

Deployment diversification
: Inference targets now span cloud GPUs, consumer GPUs, laptops, smartphones, browsers, and embedded edge boards, making model choice inseparable from hardware choice.

Figure 9 gives a size-proportional overview of the inventory by developer and family, and Figure 10 tracks the entry of new developers into the open-SLM space.

B.2 Representative Family Reference

B.3 Classification Conventions

The inventory drafts used the following practical conventions, which also informed the model-selection narrative in this paper:

- Dense models at roughly 20B parameters or below were treated as SLMs when they remained plausibly deployable on modest hardware.
- Mixture-of-Experts models were interpreted using *active* parameters for deployment reasoning, because active parameters better capture per-token compute and memory cost.
- Closed API models with undisclosed parameter counts were treated as frontier LLM baselines rather than direct members of the evaluated SLM cohort.

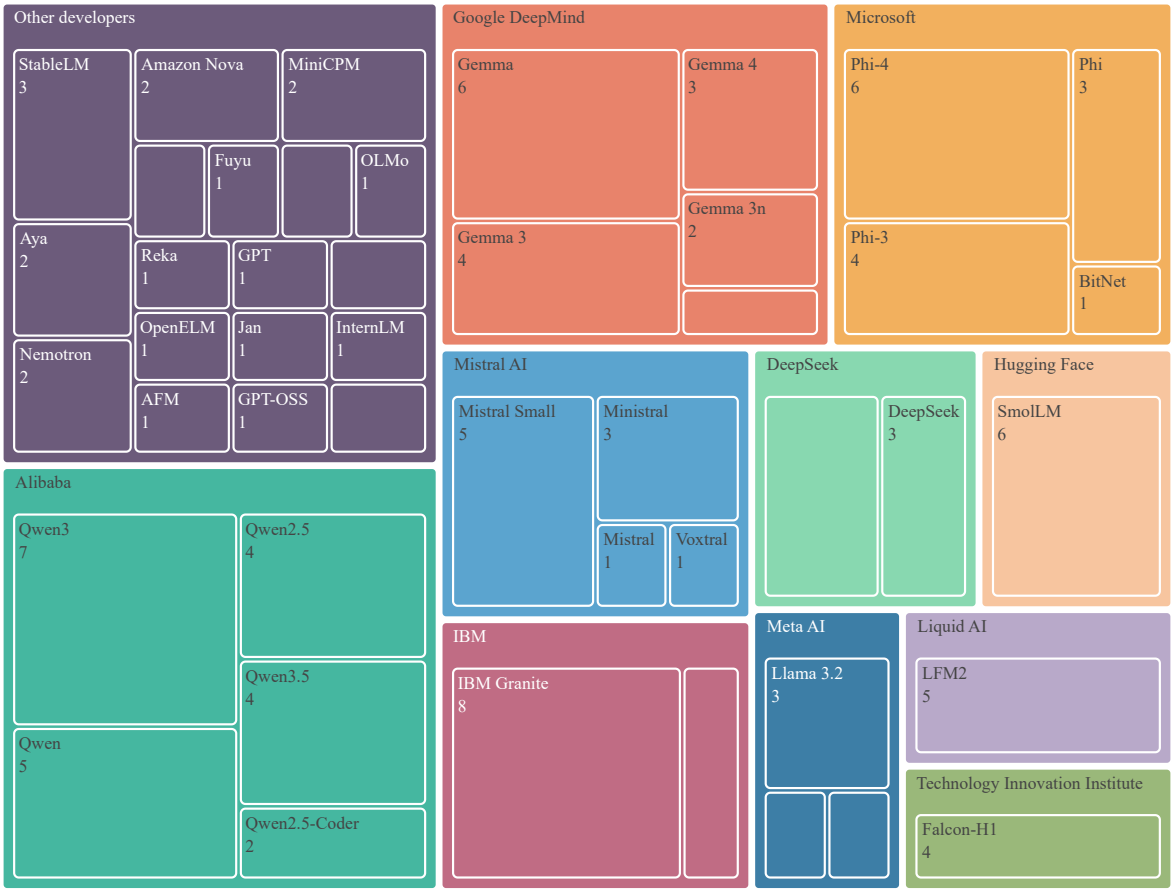


Figure 9. Documented SLM variants grouped by developer and family; area is proportional to the number of variants. Generated from `merged_llm_database.json` (snapshot September 2026).

B.4 Generated Variant Inventory

The following inventory is generated directly from `merged_llm_database.json` using `scripts/generate_model_tables.py`. The database covers 123 SLM variants and 238 LLM variants (361 entries in total, snapshot September 2026); only the SLM inventory is reproduced here, organized chronologically by release year and then by family, with parameter size as an explicit comparison column. The corresponding LLM inventory is available in the source database.

Chronology. SLM inventory: 2026, 2025, 2024, 2023 and earlier.

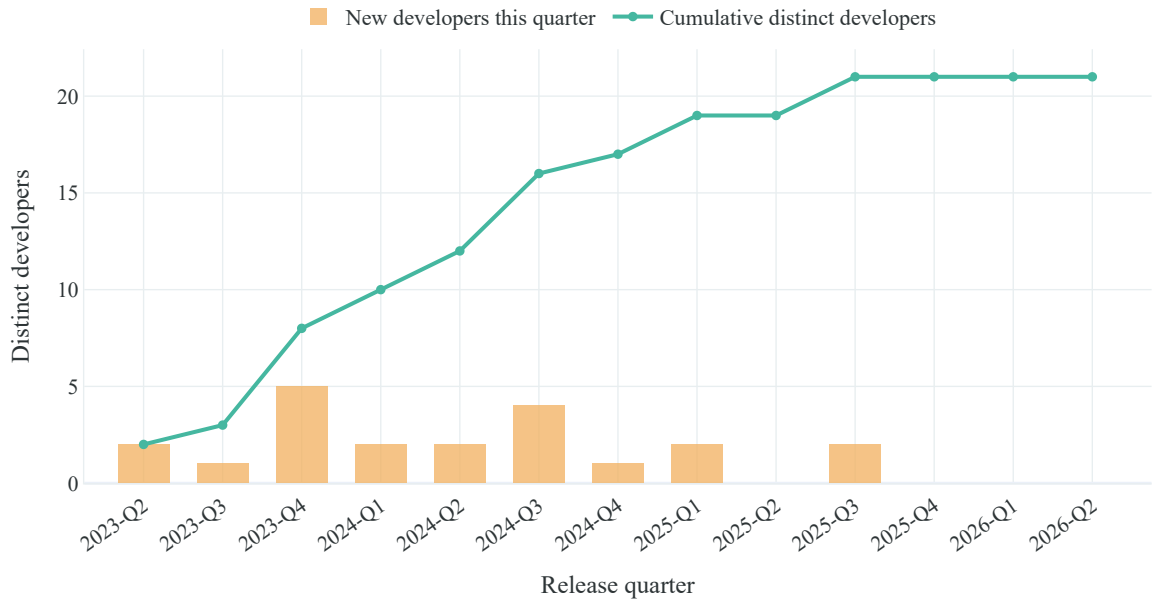


Figure 10. Cumulative distinct developers releasing documented SLM variants (line) and new developers per quarter (bars).

Table 17. Representative model families tracked in the inventory drafts.

Family	Developer	Representative Versions	Access	Typical Role
Qwen2.5 / Qwen3	Alibaba	0.5B–14B dense, MoE variants	Open weights	Strong multilingual, math, and code performance
Phi-3 / Phi-4	Microsoft	3.8B mini variants, 14B reasoning	Open weights	Data-curated reasoning-focused SLMs
Gemma 2 / 3 / 3n	Google DeepMind	1B–12B dense, mobile-first multimodal variants	Open weights	Distilled compact models with strong safety tuning
Llama 3.2 / 4	Meta	1B–3B edge models, larger MoE frontier models	Open weights	Broad deployment ecosystem from edge to server
SmolLM2 / SmolLM3	Hugging Face	135M–3B	Open weights	Lightweight, fully open research and edge baselines
DeepSeek distills	DeepSeek	1.5B–14B distilled reasoning models	Open weights	Reasoning transfer into self-hosted sizes
Granite 3 / 4	IBM	2–8B dense; 4.0 hybrid Mamba-2/MoE (350M–32B, 1–9B active)	Open weights	Enterprise-friendly, tool-use and RAG focus
Falcon-H1	TII	0.5–34B parallel attention/Mamba-2 hybrid	Open weights	Long-context edge-to-server
LFM2	Liquid AI	350M–8.3B hybrid conv/attention	Open weights	Fastest-on-CPU on-device models
MiniCPM	OpenBMB	Compact multimodal variants	Open weights	Edge multimodal use cases
GPT-5.x / Claude 4.x / Gemini 3.x	OpenAI / Anthropic / Google	API frontier models	API only	High-end proprietary baselines and teacher models
Grok / Kimi / GLM / Nova	xAI / Moonshot / Zhipu / AWS	Large API or MoE systems	Mixed	Frontier alternatives with long context or agent focus

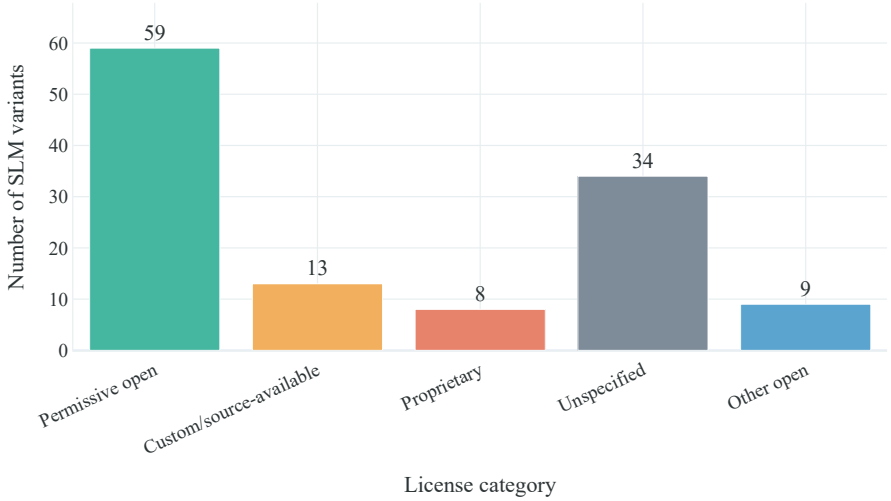


Figure 11. License posture of documented SLM variants. Permissive open licenses (Apache-2.0, MIT) account for the plurality; "unspecified" entries are typically research checkpoints without an attached license file.

Table 18. Generated chronological small language model inventory from `merged_llm_database.json`, grouped by release year and family.

Model	Access	Params	Context	Modalities	License	Release	Notes
2026							
Qwen3.5 — Alibaba (Qwen3.5 (Feb 2026)) [family page]							
Qwen3.5-0.8B	OW	0.8B	256K	Text+Vision	Apache 2.0	2026-02-16	ultra-lightweight multimodal; on-device
Qwen3.5-2B	OW	2B	256K	Text+Vision	Apache 2.0	2026-02-16	on-device multimodal chat; edge AI
Qwen3.5-4B	OW	4B	256K	Text+Vision	Apache 2.0	2026-02-16	mobile multimodal assistant; agentic coding
Qwen3.5-9B	OW	9B	256K	Text+Vision	Apache 2.0	2026-02-16	strong multimodal SLM; coding
Gemma 4 — Google DeepMind							
Gemma 4 E2B	OW	~5B (~2B effective act.)	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	on-device; multimodal
Gemma 4 E4B	OW	~8B (~4B effective act.)	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Gemma 4 26B (MoE)	OW	26B (undisclosed act.)	128K	Text+Vision	Apache 2.0	2026-04-02	reasoning; multimodal
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral Small 4 (119B)	Both	119B (6.5B act.)	256K	Text+Vision	Apache 2.0	2026-03-26	hybrid reasoning; coding
NVIDIA Nemotron — NVIDIA (Nemotron 3 Super (Mar 2026)) [family page]							
Nemotron 3 VoiceChat	OW	~12B	128K	Text+Audio	NVIDIA Custom	2026-03	voice interaction; on-device speech
2025							
Qwen3 — Alibaba (Qwen3 (Apr 2025)) [family page]							
Qwen3-4B-Instruct-2507	OW	4B	128K	Text	Apache 2.0	2025-07	improved instruction following; non-thinking mode optimised
Qwen3-0.6B	OW	0.6B	32K	Text	Apache 2.0	2025-04-29	edge inference; IoT
Qwen3-1.7B	OW	1.7B	32K	Text	Apache 2.0	2025-04-29	mobile inference; Q&A
Qwen3-30B-A3B	OW	30B (3B act.)	128K	Text	Apache 2.0	2025-04-29	MoE efficiency; broad capability in tiny footprint
Qwen3-4B-Instruct	OW	4B	128K	Text	Apache 2.0	2025-04-29	on-device chat; coding
Qwen3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04-29	single-GPU inference; reasoning
Qwen3-14B-Instruct	OW	14B	128K	Text	Apache 2.0	2025-04-29	high-quality on-premise chat; coding
DeepSeek R1-Distill — DeepSeek (R1-Distill (Jan 2025)) [family page]							
DeepSeek-R1-Distill-Qwen-1.5B	OW	1.5B	128K	Text	MIT	2025-01-20	reasoning at ultra-small scale; math
DeepSeek-R1-Distill-Qwen-7B	OW	7B	128K	Text	MIT	2025-01-20	chain-of-thought reasoning; coding
DeepSeek-R1-Distill-Llama-8B	OW	8B	128K	Text	MIT	2025-01-20	reasoning on Llama 3 base; on-premise
DeepSeek-R1-Distill-Qwen-14B	OW	14B	128K	Text	MIT	2025-01-20	strong reasoning 14B; top distill for coding
Gemma 3 — Google DeepMind (Gemma 3 (Mar 2025)) [family page]							
Gemma 3 270M-IT	OW	270M	32K	Text	Google Gemma License	2025-03	ultra-lightweight prototyping; experiments
Gemma 3 1B-IT	OW	1B	32K	Text	Google Gemma License	2025-03	on-device mobile inference; lightweight Q&A

continued on next page

Table 18. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Gemma 3 4B-IT	OW	4B	128K	Text+Vision	Google Gemma License	2025-03	multimodal on-device; image understanding
Gemma 3 12B-IT	OW	12B	128K	Text+Vision	Google Gemma License	2025-03	high-quality assistant; 140+ languages
Gemma 3n — Google DeepMind (Gemma-3n (2025)) [family page]							
Gemma-3n-E2B-IT	OW	~5B (~2B effective act.)	32K	Text+Vision+Audio+Video	Google Gemma License	2025	mobile-first multimodal; on-device speech/vision/text
Gemma-3n-E4B-IT	OW	~8B (~4B effective act.)	32K	Text+Vision+Audio+Video	Google Gemma License	2025	higher-accuracy mobile multimodal; edge devices
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM3-3B	OW	3B	64K	Text	Apache 2.0	2025	dual-mode reasoning; tool calling
Granite — IBM (Granite) [family page]							
Granite 3.2-2B	OW	2B	125K	Text	See source	2025-02-26	Thinking mode; 128K context
Granite 3.2-8B	OW	8B	125K	Text	See source	2025-02-26	Thinking mode (chain-of-thought); 128K context
IBM Granite — IBM (Granite 3.3 (Apr 2025)) [family page]							
Granite 4.0 Nano 350M	OW	350M	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 Nano 1B	OW	1B	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 H-Tiny	OW	7B (1B act.)	128K	Text	Apache 2.0	2025-10-02	low-latency; agentic
Granite 4.0 H-Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; tool use
Granite 4.0 Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; RAG
Granite 4.0 H-Small	OW	32B (9B act.)	128K	Text	Apache 2.0	2025-10-02	enterprise; agentic
Granite 3.3-2B-Instruct	OW	2B	128K	Text	Apache 2.0	2025-04	enterprise NLP; governed AI
Granite 3.3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04	enterprise reasoning; Fill-in-Middle code
Jan — Jan.ai (Jan-v1-4B (2025)) [family page]							
Jan-v1-4B	OW	4B	128K	Text	Apache 2.0	2025	agentic reasoning; tool use
LFM2 — Liquid AI							
LFM2-8B-A1.5B (MoE)	OW	8.3B (1.5B act.)	32K	Text	LFM Open License v1.0	2025-08	on-device; agentic
LFM2-2.6B	OW	2.6B	32K	Text	LFM Open License v1.0	2025-08	on-device
LFM2-350M	OW	350M	32K	Text	LFM Open License v1.0	2025-07-10	on-device; low latency
LFM2-700M	OW	700M	32K	Text	LFM Open License v1.0	2025-07-10	on-device
LFM2-1.2B	OW	1.2B	32K	Text	LFM Open License v1.0	2025-07-10	on-device; agentic
Llama 4 — Meta AI (Llama 4 Maverick (Apr 2025)) [family page]							
Llama 4 Scout	Both	109B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	extreme long-context; single H100 runnable
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4-mini-flash-reasoning	OW	3.8B	128K	Text	MIT	2025-04	fast reasoning; edge agents
Phi-4-mini-reasoning	OW	3.8B	128K	Text	MIT	2025-04	chain-of-thought reasoning; math
Phi-4-reasoning	OW	14B	128K	Text	MIT	2025-04	complex reasoning; STEM

continued on next page

Table 18. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Phi-4-reasoning-plus	OW	14B	128K	Text	MIT	2025-04	RL-trained reasoning; math olympiad-level
Phi-4-mini-instruct	OW	3.8B	128K	Text	MIT	2025-02	on-device chat; function calling
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral 3 (14B)	OW	14B	–	Text+Vision	See source	2025-12-02	edge deployment; multimodal
Ministral-3B-Instruct-2512	OW	3.8B	256K	Text+Vision	Apache 2.0	2025-12	tiny multimodal; function calling
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral Small 3.2 (24B)	Both	24B	128K	Text+Vision	Apache 2.0	2025-06	updated multimodal small model; RAG backend
Mistral Small 3.1 (24B)	Both	24B	128K	Text+Vision	Apache 2.0	2025-03	enterprise RAG; reasoning
Voxtral — Mistral AI (Voxtral) [family page]							
Voxtral Mini	API	undisclosed	–	Text+Audio	Proprietary	2025-09-15	speech understanding; transcription
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron-2-Nano-8B	OW	8B	–	Text	See source	2025-03-18	on-device; efficient
GPT-OSS — OpenAI (gpt-oss-120b (2025)) [family page]							
gpt-oss-20b	OW	21B (3.6B act.)	128K	Text	Apache 2.0	2025	edge reasoning; 16GB device inference
MiniCPM — OpenBMB (MiniCPM-V 4.5 (2025)) [family page]							
MiniCPM-V 4.5	OW	~8B	32K	Text+Vision	Apache 2.0	2025	mobile vision-language; on-device VQA
MiniCPM-o 3.0	OW	~9B	32K	Text+Vision+Audio	Apache 2.0	2025	end-to-end multimodal on-device; live streaming
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM3-8B	OW	8B	32K	Text	See source	2025-01-09	Reasoning via deep thinking; Knowledge-rich training pipeline
Falcon-H1 — Technology Innovation Institute							
Falcon-H1-0.5B	OW	0.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-1.5B	OW	1.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-3B	OW	3B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
Falcon-H1-7B	OW	7B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
2024							
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Coder-1.5B	OW	1.5B	–	Text	See source	2024-11-12	compact coding model
Qwen2-0.5B	OW	0.5B	–	Text	See source	2024-06-07	active
Qwen2-1.5B	OW	1.5B	–	Text	See source	2024-06-07	active
Qwen1.5-0.5B	OW	0.5B	–	Text	See source	2024-02-05	legacy
Qwen2.5 — Alibaba (Qwen2.5 (Sep 2024)) [family page]							
Qwen2.5-0.5B-Instruct	OW	0.5B	32K	Text	Apache 2.0	2024-09-19	extreme edge/IoT; classification
Qwen2.5-1.5B-Instruct	OW	1.5B	32K	Text	Apache 2.0	2024-09-19	edge NLP; lightweight assistants

continued on next page

Table 18. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Qwen2.5-3B-Instruct	OW	3B	32K	Text	Apache 2.0	2024-09-19	mobile chat; summarization
Qwen2.5-7B-Instruct	OW	7B	128K	Text	Apache 2.0	2024-09-19	general assistant; coding
Qwen2.5-Coder — Alibaba (Qwen2.5-Coder (Nov 2024)) [family page]							
Qwen2.5-Coder-7B-Instruct	OW	7B	128K	Text+Code	Apache 2.0	2024-11	code generation; debugging
Qwen2.5-Coder-14B-Instruct	OW	14B	128K	Text+Code	Apache 2.0	2024-11	advanced code assistant; agentic code tasks
OLMo — Allen Institute for AI (Ai2) (OLMo) [family page]							
OLMo-1B	OW	1B	2K	Text	See source	2024-02-01	Fully open (data, code, weights, logs); Dolma training data
Amazon Nova — Amazon (Nova Premier (2025)) [family page]							
Amazon Nova Lite	API	Undisclosed	300K	Text+Vision	Proprietary (AWS)	2024-12-03	multimodal understanding; cost-efficient AWS inference
Amazon Nova Micro	API	Undisclosed	128K	Text	Proprietary (AWS)	2024-12-03	AWS-native low-latency text tasks; Lambda AI apps
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device)	API	~3B	4K	Text	Proprietary	2024-06-10	On-device Apple Intelligence; Privacy-preserving
OpenELM — Apple (OpenELM) [family page]							
OpenELM	OW	3B	–	Text	See source	2024-04-24	layer-wise scaling; on-device
Aya — Cohere (Aya) [family page]							
Aya Expanse-8B	OW	8B	8K	Text	See source	2024-10-22	Improved multilingual performance; Data arbitrage and synthetic data strategies
Aya 23-8B	OW	8B	8K	Text	See source	2024-05-23	23 language support; Multilingual instruction following
Cohere Command — Cohere (Command A (Mar 2025)) [family page]							
Command R7B	Both	7B	128K	Text	Apache 2.0	2024-12	RAG; multilingual enterprise tasks
TinyLlama — Community (TinyLlama-1.1B (2024)) [family page]							
TinyLlama-1.1B-Chat	OW	1.1B	2K	Text	Apache 2.0	2024	ultra-light on-device chat; prototyping
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-VL-7B	OW	7B	4K	Text+Vision	See source	2024-03-13	Vision-language model; Hybrid vision encoder
DeepSeek-Math-7B	OW	7B	4K	Text	See source	2024-02-05	Math-specialized; Group Relative Policy Optimization (GRPO)
Gemma — Google DeepMind (Gemma) [family page]							
Gemma 2-2B	OW	2B	8K	Text	See source	2024-07-31	Knowledge distillation from larger model; Sliding window attention
Gemma 2-9B	OW	9B	–	Text	See source	2024-06-27	improved efficiency; knowledge distillation
PaliGemma	OW	3B	8K	Text+Vision	See source	2024-05-14	Vision-language model; SigLIP vision encoder + Gemma 2B

continued on next page

Table 18. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
RecurrentGemma-2B	OW	2B	8K	Text	See source	2024-04-09	Griffin architecture with linear recurrence; Efficient inference
Gemma-2B	OW	2B	–	Text	See source	2024-02-21	legacy
Gemma-7B	OW	7B	–	Text	See source	2024-02-21	built from Gemini technology; efficient
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM-135M	OW	135M	2K	Text	See source	2024-07-17	Ultra-small LM; On-device capable
SmolLM	OW	1.7B	–	Text	See source	2024-07-16	tiny but capable; on-device
SmolLM2-135M	OW	135M	2K	Text	Apache 2.0	2024	ultra-micro inference; browser/WASM
SmolLM2-360M-Instruct	OW	360M	8K	Text	Apache 2.0	2024	on-device/browser inference
SmolLM2-1.7B-Instruct	OW	1.7B	8K	Text	Apache 2.0	2024	efficient on-device chat; coding assist
Llama 3.1 — Meta AI (Llama 3.1 (Jul 2024)) [family page]							
Llama 3.1-8B-Instruct	OW	8B	128K	Text	Llama 3.1 Community License	2024-07-23	most fine-tuned SLM base globally; RAG
Llama 3.2 — Meta AI (Llama 3.2 (Sep 2024)) [family page]							
Llama 3.2-1B-Instruct	OW	1B	128K	Text	Llama 3.2 Community License	2024-09-25	ultra-small on-device; Android/iOS apps
Llama 3.2-3B-Instruct	OW	3B	128K	Text	Llama 3.2 Community License	2024-09-25	mobile AI; default on-device Qualcomm
Llama 3.2-11B-Vision-Instruct	OW	11B	128K	Text+Vision	Llama 3.2 Community License	2024-09-25	vision-language tasks; document understanding
BitNet — Microsoft (BitNet) [family page]							
BitNet b1.58	OW	3B	–	Text	See source	2024-02-27	1-bit LLM (ternary weights: -1, 0, 1); Extreme efficiency
Phi-3 — Microsoft (Phi-3.5 (Aug 2024)) [family page]							
Phi-3.5-mini-instruct	OW	3.8B	128K	Text	MIT	2024-08	multilingual instruction following
Phi-3-mini-128k-instruct	OW	3.8B	128K	Text	MIT	2024-04-23	on-device chat; instruction following
Phi-3-small-128k-instruct	OW	7B	128K	Text	MIT	2024-04-23	balanced reasoning & coding
Phi-3-medium-128k-instruct	OW	14B	128K	Text	MIT	2024-04-23	on-premise deployment; complex reasoning
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4	OW	14B	16K	Text	MIT	2024-12-12	STEM reasoning; math
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral-8B-Instruct-2410	OW	8B	128K	Text	Apache 2.0	2024-10	edge-friendly assistant; function calling
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral NeMo 12B	OW	12B	128K	Text	Apache 2.0	2024-07	enterprise RAG; long-document summarization
Nemotron — NVIDIA (Nemotron) [family page]							
Minitron-8B	OW	8B	8K	Text	See source	2024-07-17	Width and depth pruning; Knowledge distillation

continued on next page

Table 18. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
GPT — OpenAI (GPT) [family page]							
GPT-4o mini	API	undisclosed	–	Text+Vision	Proprietary	2024-07-18	cost-efficient; fast
Reka — Reka AI (Reka) [family page]							
Reka Edge	API	7B	–	Text+Vision	Proprietary	2024-02-12	compact; multimodal
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM 2 12B	OW	12B	4K	Text	See source	2024-02-20	Multilingual (7 languages); Strong code capability
StableLM 2 1.6B	OW	1.6B	4K	Text	Stability AI License	2024-01-19	lightweight creative writing; basic chat
2023 and earlier							
Fuyu — Adept AI (Fuyu) [family page]							
Fuyu-8B	OW	8B	–	Text+Vision	See source	2023-10-17	Simplified multimodal architecture; No separate vision encoder
Qwen — Alibaba (Qwen) [family page]							
Qwen-Audio	OW	7B	–	Text+Audio	See source	2023-11-30	Audio understanding; Multi-task audio processing
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-Coder-1.3B	OW	1.3B	–	Text	See source	2023-11-02	compact code model; fill-in-the-middle
Gemini — Google DeepMind (Gemini) [family page]							
Gemini 1.0 Nano	API	3.25B	–	Text	Proprietary	2023-12-06	on-device; efficient
Phi — Microsoft (Phi) [family page]							
Phi-2	OW	2.7B	–	Text	See source	2023-12-13	language comprehension; reasoning
Phi-1.5	OW	1.3B	–	Text	See source	2023-09-11	reasoning; language understanding
Phi-1	OW	1.3B	–	Text	See source	2023-06-20	Python coding; textbook-quality data
Mistral — Mistral AI (Mistral) [family page]							
Mistral Tiny (Mistral-7B-Instruct)	API	7B	32K	Text	Proprietary	2023-12-11	API-served Mistral 7B Instruct; Cost-effective API option
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral 7B v0.3	OW	7.3B	32K	Text	Apache 2.0	2023-09-27	general chat; RAG
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM-Zephyr-3B	OW	3B	–	Text	See source	2023-11-29	chat assistant; compact
RedPajama — Together AI (RedPajama) [family page]							
RedPajama-INCITE-7B	OW	7B	2K	Text	See source	2023-05-05	Fully open training data; Reproducible

B.5 Monitoring Sources and Maintenance

Table 19. High-value monitoring sources retained from the inventory files.

Source	Use
Hugging Face model hub and leaderboards	New releases, model cards, and benchmark submissions
Artificial Analysis	Context windows, pricing, and comparative capability snapshots
LLM Stats	Consolidated model metadata and API comparisons
LMSYS / Chatbot Arena	Human preference rankings
BentoML and vendor engineering blogs	Practical deployment write-ups and model updates
Official vendor changelogs	Release confirmation for OpenAI, Anthropic, Google, Meta, Mistral, Alibaba, and DeepSeek

Recommended maintenance queries from the inventory drafts included combinations such as “best small language model [year]”, “new open-weight reasoning model”, and vendor-specific searches around context window, model card, and release notes. In practice, the highest-signal workflow is to combine vendor documentation with model cards and a benchmark tracker rather than relying on any single static list.

References

1. OpenAI. GPT-4 Technical Report. arXiv preprint arXiv:2303.08774 2023.
2. Anthropic. The Claude Model Family. Anthropic Technical Report 2024.
3. Gemini Team, Anil R, Borgeaud S, et al. Gemini: A Family of Highly Capable Multimodal Models. arXiv preprint arXiv:2312.11805 2023.
4. Grattafiori A et al. The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783 2024.
5. Brown TB, Mann B, Ryder N, et al. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems 2020;33:1877–901.
6. Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models. arXiv preprint arXiv:2001.08361 2020.
7. Hoffmann J, Borgeaud S, Mensch A, et al. Training Compute-Optimal Large Language Models. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022:30016–30. URL: https://arxiv.org/abs/2203.15556.
8. Patterson D, Gonzalez J, Le Q, et al. Carbon Emissions and Large Neural Network Training. arXiv preprint arXiv:2104.10350 2021.
9. Vries A de. The Growing Energy Footprint of Artificial Intelligence. Joule 2023;7:2191–4.
10. Sanh V, Debut L, Chaumond J, and Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108 2019.
11. Jiao X, Yin Y, Shang L, Jiang X, Chen X, Li L, Wang F, and Liu Q. TinyBERT: Distilling BERT for Natural Language Understanding. 2020.
12. Abdin M, Aneja J, Awadalla H, et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 2024.
13. Gemma Team, Riviere M, Pathak S, et al. Gemma: Open Models Based on Gemini Research and Technology. arXiv preprint arXiv:2403.08295 2024.
14. Qwen Team, Yang A, Yang B, et al. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 2024.
15. MIT Technology Review. 10 Breakthrough Technologies 2025. MIT Technology Review 2025.
16. Technology Innovation Institute. Falcon-H1: A Family of Hybrid Mamba-Transformer Language Models. arXiv preprint arXiv:2506.07086 2025.
17. Qwen Team. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388 2025.
18. Gemma Team, Google DeepMind. Gemma 4 Technical Report. arXiv preprint arXiv:2607.02770 2026.
19. Apple. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv preprint arXiv:2507.13575 2025.

20. Belcak P, Heinrich G, Diao S, Fu Y, Dong X, Muralidharan S, Lin YC, and Molchanov P. Small Language Models are the Future of Agentic AI. arXiv preprint arXiv:2506.02153 2025.

21. Chen L and Varoquaux G. What is the Role of Small Models in the LLM Era: A Survey. arXiv preprint arXiv:2409.06857 2024.

22. Chen Y, Zhao J, and Han H. A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv preprint arXiv:2505.07460 2025.

23. Lu Z, Li X, Cai D, Yi R, Liu F, Zhang X, Lane ND, and Xu M. Small Language Models: Survey, Measurements, and Insights. arXiv preprint arXiv:2409.15790 2024.

24. Nguyen CV, Shen X, Aponte R, Xia Y, Basu S, Hu Z, Chen J, Parmar M, Kunapuli S, Barrow J, Wu J, Singh A, Wang Y, Gu J, Dernoncourt F, Ahmed NK, Lipka N, Zhang R, Chen X, Yu T, Kim S, Deilamsalehy H, Park N, Rimer M, Zhang Z, Yang H, Rossi RA, and Nguyen TH. A Survey of Small Language Models. arXiv preprint arXiv:2410.20011 2024.

25. Wang F, Zhang Z, Zhang X, Wu Z, Mo T, Lu Q, Wang W, Li R, Xu J, Tang X, He Q, Ma Y, Huang M, and Wang S. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. arXiv preprint arXiv:2411.03350 2024.

26. Zhou Z, Kurz S, and Zhao Z. Revisiting Pruning vs Quantization for Small Language Models. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China, 2025:12055–70. URL: https://aclanthology.org/2025.findings-emnlp.645/.

27. Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017:5998–6008. URL: https://arxiv.org/abs/1706.03762.

28. Devlin J, Chang MW, Lee K, and Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of NAACL*. 2019:4171–86. URL: https://arxiv.org/abs/1810.04805.

29. Radford A, Wu J, Child R, Luan D, Amodei D, and Sutskever I. Language Models are Unsupervised Multitask Learners. OpenAI Blog 2019.

30. Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways. Journal of Machine Learning Research 2023;24:1–113.

31. Touvron H, Lavril T, Izacard G, et al. LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971 2023.

32. Touvron H, Martin L, Stone K, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv preprint arXiv:2307.09288 2023.

33. Almazrouei E, Alobeidli H, Alshamsi A, et al. The Falcon Series of Open Language Models. arXiv preprint arXiv:2311.16867 2023.

34. Gunasekar S, Zhang Y, Aneja J, Mendes CCT, Giorno AD, Gopi S, Javaheripi M, Kauffmann P, Rosa G de, Saarikivi O, et al. Textbooks Are All You Need. arXiv preprint arXiv:2306.11644 2023.

35. Microsoft Research. Phi-2: The Surprising Power of Small Language Models. Microsoft Research Blog 2023.

36. Gemma Team, Riviere M, Pathak S, et al. Gemma 2: Improving Open Language Models at a Practical Size. arXiv preprint arXiv:2408.00118 2024.

37. Microsoft Research. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv preprint arXiv:2503.01743 2025.

38. Hinton G, Vinyals O, and Dean J. Distilling the Knowledge in a Neural Network. arXiv preprint arXiv:1503.02531 2015.

39. Gemma Team et al. Gemma 3 Technical Report. arXiv preprint arXiv:2503.19786 2025.

40. Romero A, Ballas N, Kahou SE, et al. FitNets: Hints for Thin Deep Nets. arXiv preprint arXiv:1412.6550 2015.

41. Mitra A, Del Corro L, et al. Orca 2: Teaching Small Language Models How to Reason. arXiv preprint arXiv:2311.11045 2023.

42. Abdin M et al. Phi-4 Technical Report. arXiv preprint arXiv:2412.08905 2024.

43. Frantar E, Ashkboos S, Hoefler T, and Alistarh D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In: *Proceedings of the International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2210.17323.
44. Lin J, Tang J, Tang H, et al. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In: *Proceedings of Machine Learning and Systems*. 2024. URL: https://arxiv.org/abs/2306.00978.
45. Gerganov G. llama.cpp. GitHub repository: https://github.com/ggerganov/llama.cpp. 2023.
46. Kim S, Hooper C, et al. SqueezeLLM: Dense-and-Sparse Quantization. arXiv preprint arXiv:2306.07629 2024.
47. Frantar E and Alistarh D. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In: *Proceedings of the International Conference on Machine Learning*. 2023:10323–37. URL: https://arxiv.org/abs/2301.00774.
48. Hu EJ, Shen Y, Wallis P, et al. LoRA: Low-Rank Adaptation of Large Language Models. In: *Proceedings of the International Conference on Learning Representations*. 2022. URL: https://arxiv.org/abs/2106.09685.
49. Dettmers T, Pagnoni A, Holtzman A, and Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: https://arxiv.org/abs/2305.14314.
50. Abdin M et al. Phi-3.5-MoE: Mixture of Experts for Efficient Language Models. Microsoft Research Technical Report 2024.
51. Abdin M et al. Phi-4-Reasoning Technical Report. arXiv preprint arXiv:2504.21318 2025.
52. Yang A, Yang B, Hui B, et al. Qwen2 Technical Report. arXiv preprint arXiv:2407.10671 2024.
53. Zhang P, Zeng G, Wang T, and Lu W. TinyLlama: An Open-Source Small Language Model. arXiv preprint arXiv:2401.02385 2024.
54. Jiang AQ, Sablayrolles A, Mensch A, et al. Mistral 7B. arXiv preprint arXiv:2310.06825 2023.
55. Allal LB, Lozhkov A, Penedo G, et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv preprint arXiv:2502.02737 2025.
56. Singer P, Gissin D, et al. H2O-Danube-1.8B Technical Report. arXiv preprint arXiv:2401.16818 2024.
57. AI2 Team. OLMo 2: Open Language Model. arXiv preprint arXiv:2501.00656 2025.
58. Stability AI. StableLM 2 1.6B Technical Report. Stability AI Technical Report 2024.
59. BigScience Workshop. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv preprint arXiv:2211.05100 2022.
60. Nguyen T and Nguyen T. An Evaluation of LLMs Inference on Popular Single-board Computers. arXiv preprint arXiv:2511.07425 2025.
61. Meta AI. ExecuTorch: End-to-End Solution for Enabling On-Device AI. PyTorch Blog. 2024. URL: https://pytorch.org/executorch/.
62. MLC Team. MLC LLM: Universal LLM Deployment Engine. GitHub repository: https://github.com/mlc-ai/mlc-llm. 2024.
63. Microsoft. ONNX Runtime. GitHub repository: https://github.com/microsoft/onnxruntime. 2019.
64. MLC Team. WebLLM: High-Performance In-Browser LLM Inference Engine. GitHub repository. 2024. URL: https://arxiv.org/abs/2412.15803.
65. Google Developers. LiteRT with QNN Acceleration. Google AI Blog. 2025. URL: https://ai.google.dev/edge/litert.
66. Xu J, Liu C, Tan M, et al. On-Device Language Models: A Comprehensive Review. arXiv preprint arXiv:2409.00088 2024.
67. Ong I, Almahairi A, Wu V, Chiang WL, Wu T, Gonzalez JE, Kadous MW, and Stoica I. RouteLLM: Learning to Route LLMs with Preference Data. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: https://arxiv.org/abs/2406.18665.
68. Shao C, Hu X, Lin Y, and Xu F. Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. arXiv preprint arXiv:2502.04392 2025.

69. Zhang J, Lan T, Zhu M, Liu Z, Hoang T, Kokane S, Yao W, Tan J, Prabhakar A, Chen H, Liu Z, Feng Y, Awalgaonkar T, Murthy R, Hu E, Chen Z, Xu R, Niebles JC, Heinecke S, Wang H, Savarese S, and Xiong C. xLAM: A Family of Large Action Models to Empower AI Agent Systems. arXiv preprint arXiv:2409.03215 2024.

70. Allal LB, Lozhkov A, Bakouch E, Blanco GM, Tunstall L, Lajarín AP, Marafioti A, Zakka C, Werra L von, and Wolf T. SmolLM2: Small Language Models With Exceptional Reasoning. arXiv preprint arXiv:2502.02737 2025.

71. Bakouch E, Bakouch L, Lozhkov A, Werra L von, Wolf T, et al. SmolLM3: smol, Multilingual, Long-Context Reasoner. Hugging Face blog, `https://huggingface.co/blog/smollm3`. 2025.

72. Shazeer N, Mirhoseini A, Maziarz K, Davis A, Le Q, Hinton G, and Dean J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. 2017.

73. Fedus W, Zoph B, and Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. Journal of Machine Learning Research 2022;23:1–39.

74. Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, Chaplot DS, Casas D de las, Hanna EB, Bressand F, et al. Mixtral of Experts. arXiv preprint arXiv:2401.04088 2024.

75. Dai D, Deng C, Zhao C, Xu RX, Gao H, Chen D, Li J, Zeng W, Yu X, Wu Y, et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv preprint arXiv:2401.06066 2024.

76. Meta AI. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. `https://ai.meta.com/blog/llama-4-multimodal-intelligence/`. 2025.

77. Gu A and Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv preprint arXiv:2312.00752 2023.

78. Lieber O, Lenz B, Bata H, Cohen G, Osin J, Dalmedigos I, Safahi E, Meirom S, Belinkov Y, Shalev-Shwartz S, et al. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv preprint arXiv:2403.19887 2024.

79. IBM Granite Team. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models for Enterprise. `https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models`. 2025.

80. NVIDIA. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. arXiv preprint arXiv:2508.14444 2025.

81. Qwen Team. Qwen3.5: Small Models for On-Device Applications. `https://qwenlm.github.io/blog/`. Alibaba Qwen model release, March 2026. 2026.

82. Li Y, Bubeck S, Eldan R, Giorno AD, Gunasekar S, and Lee YT. Textbooks Are All You Need II: phi-1.5 technical report. arXiv preprint arXiv:2309.05463 2023.

83. Eldan R and Li Y. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv preprint arXiv:2305.07759 2023.

84. Mukherjee S, Mitra A, Jawahar G, Agarwal S, Palangi H, and Awadallah A. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv preprint arXiv:2306.02707 2023.

85. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948 2025.

86. Xu H, Peng B, Awadalla H, Chen D, Chen YC, Gao M, Kim YJ, Li Y, Ren L, Shen Y, Wang S, Xu W, Gao J, and Chen W. Phi-4-Mini-Reasoning: Exploring the Limits of Small Reasoning Language Models in Math. arXiv preprint arXiv:2504.21233 2025.

87. Sreenivas ST, Muralidharan S, Joshi R, Chochowski M, Patwary M, Shoeybi M, Catanzaro B, Kautz J, and Molchanov P. Minitron: Efficient LLM Compression via Pruning and Knowledge Distillation. arXiv preprint arXiv:2407.14679 2024.

88. Beyer L, Steiner A, Pinto AS, Kolesnikov A, Wang X, and Zhai X. PaliGemma: A Versatile 3B VLM for Transfer. arXiv preprint arXiv:2407.07726 2024.

89. Google. Gemma 3n: On-Device Multimodal Models. Google AI Blog. 2025. URL: `https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/`.

90. Kim HH, Kang J, Bae K, et al. Small language models learn enhanced reasoning skills from medical datasets. npj Digital Medicine 2025.

91. Hasan M et al. OptimCLM: Optimizing Clinical Language Models for Predicting Patient Outcomes via Knowledge Distillation, Pruning, and Quantization. International Journal of Medical Informatics 2024.
92. Rozière B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T, et al. Code Llama: Open Foundation Models for Code. arXiv preprint arXiv:2308.12950 2023.
93. Lozhkov A, Li R, Allal LB, Cassano F, Lamy-Poirier J, Tazi N, Tang A, Pykhtar D, Liu J, Wei Y, et al. StarCoder2 and The Stack v2: The Next Generation. arXiv preprint arXiv:2402.19173 2024.
94. Liu Z et al. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv preprint arXiv:2401.14196 2024.
95. Üstün A, Aryabumi V, Yong ZX, Ko WY, D'souza D, Onilude G, Bhandari N, Singh S, Ooi HL, Kayid A, et al. Aya Model: An Instruction Finetuned Open-Access Multilingual Language Model. arXiv preprint arXiv:2402.07827 2024.
96. Yang C, Lu W, Zhu Y, Wang Y, Chen Q, Gao C, Yan B, and Chen Y. Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application. arXiv preprint arXiv:2407.01885 2024.
97. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, and Zhou D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 2022.
98. Feng X, Xu R, and Liu Y. Teaching Small Language Models Reasoning through Counterfactual Distillation. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024. URL: https://aclanthology.org/2024.emnlp-main.000.
99. Liao R, Li Q, Liu X, and Zhang Y. Neural-Symbolic Collaborative Distillation: Advancing Small Language Models for Complex Reasoning Tasks. arXiv preprint arXiv:2409.13203 2024.
100. Xu X, Li M, Tao C, Shen T, Cheng R, Li J, Xu C, Tao D, and Zhou T. A Survey on Knowledge Distillation of Large Language Models. arXiv preprint arXiv:2402.13116 2024.
101. Ballout M, Tresp V, Divo F, and Kersting K. Efficient Knowledge Distillation: Empowering Small Language Models with Teacher Model Insights. arXiv preprint arXiv:2409.12586 2024.
102. Fu Y, Shi Q, and Yin J. Dynamic Self-Distillation via Previous Mini-batches for Fine-tuning Small Language Models. arXiv preprint arXiv:2411.16991 2024.
103. Sun M, Liu Z, Bair A, and Kolter JZ. A Simple and Effective Pruning Approach for Large Language Models. 2024.
104. Hendrycks D, Burns C, Basart S, et al. Measuring Massive Multitask Language Understanding. Proceedings of the International Conference on Learning Representations 2021.
105. Zellers R, Holtzman A, Bisk Y, et al. HellaSwag: Can a Machine Really Finish Your Sentence? In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 2019. URL: https://arxiv.org/abs/1905.07830.
106. Sakaguchi K, Le Bras R, Bhagavatula C, and Choi Y. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. Communications of the ACM 2021;64:99–106.
107. Clark P, Cowhey I, Etzioni O, et al. Think You Have Solved Question Answering? Try ARC. arXiv preprint arXiv:1803.05457 2018.
108. Bisk Y, Zellers R, et al. PIQA: Reasoning about Physical Intuition in Natural Language. Proceedings of AAAI 2020.
109. Clark C, Lee K, Chang MW, Kwiatkowski T, Collins M, and Toutanova K. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. Proceedings of NAACL 2019.
110. Cobbe K, Kosaraju V, Bavarian M, et al. Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:2110.14168 2021.
111. Hendrycks D, Burns C, Kadavath S, et al. Measuring Mathematical Problem Solving With the MATH Dataset. Proceedings of NeurIPS 2021.
112. Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374 2021.
113. Austin J, Odena A, Nye M, et al. Program Synthesis with Large Language Models. arXiv preprint arXiv:2108.07732 2021.

114. Williams A, Nangia N, and Bowman SR. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. Proceedings of NAACL 2018.

115. Nie Y, Williams A, Dinan E, et al. Adversarial NLI: A New Benchmark for Natural Language Understanding. Proceedings of the Annual Meeting of the Association for Computational Linguistics 2020.

116. Joshi M, Choi E, Weld DS, and Zettlemoyer L. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. Proceedings of the Annual Meeting of the Association for Computational Linguistics 2017.

117. Rajpurkar P, Jia R, and Liang P. Know What You Don't Know: Unanswerable Questions for SQuAD. Proceedings of the Annual Meeting of the Association for Computational Linguistics 2018.

118. Hermann KM, Kocisky T, et al. Teaching Machines to Read and Comprehend. Advances in Neural Information Processing Systems 2015.

119. Narayan S, Cohen SB, and Lapata M. Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. Proceedings of EMNLP 2018.

120. Zheng L, Chiang WL, Sheng Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 2023.

121. Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. Proceedings of SOSP 2023.

122. Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, et al. Holistic Evaluation of Language Models. Transactions on Machine Learning Research 2023. arXiv:2211.09110.

123. Gao L, Tow J, Abbasi B, et al. A Framework for Few-Shot Language Model Evaluation. Zenodo. 2023. URL: https://github.com/EleutherAI/lm-evaluation-harness.

124. Wang J, Zeng Y, Guo J, Ma Y, Liu A, and Liu X. SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment. arXiv preprint arXiv:2511.13023 2025.

125. Apple Inc. Apple Intelligence Foundation Language Models. Tech. rep. Apple Machine Learning Research, 2024. URL: https://arxiv.org/abs/2407.21075.

126. Apple Machine Learning Research. Introducing the Third Generation of Apple's Foundation Models. https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models. 2026.

127. Samsung Research. Samsung Gauss: Language Model for Everyday Life. Samsung AI Report 2023.

128. Apple ML Research. MLX: An Array Framework for Apple Silicon. GitHub Repository 2023.

129. NVIDIA Corporation. TensorRT-LLM: Accelerating Large Language Model Inference. NVIDIA Developer Blog 2024.

130. Lu Z, Li X, Cai D, Yi R, Liu F, Liu W, Luan J, Zhang X, Lane ND, and Xu M. Demystifying Small Language Models for Edge Deployment. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria, 2025:14747–64. URL: https://aclanthology.org/2025.acl-long.718/.

131. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih Wt, Rocktäschel T, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020.

132. Mistral AI. Codestral: Hello, World! Mistral AI Blog 2024.

133. Qi X, Zeng Y, et al. Fine-tuning Aligned Language Models Compromises Safety. arXiv preprint arXiv:2310.03693 2023.

134. Blodgett SL, Barocas S, III HD, and Wallach H. Language (Technology) is Power: A Critical Survey of "Bias" in NLP. 2020.

135. Zhang Y, Li Y, Cui L, Cai D, Liu L, Fu T, Huang X, Zhao E, Zhang Y, Chen Y, et al. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. arXiv preprint arXiv:2309.01219 2023.

136. Leviathan Y, Kalman M, and Matias Y. Fast Inference from Transformers via Speculative Decoding. 2023.

137. Qualcomm Technologies. On-Device AI: Enabling the Next Generation of Mobile Intelligence. Qualcomm White Paper 2024.

138. Strubell E, Ganesh A, and McCallum A. Energy and Policy Considerations for Deep Learning in NLP. Proceedings of ACL 2019:3645–50.

139. Patterson D, Gonzalez J, Hölzle U, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, and Dean J. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. IEEE Computer 2022.

140. Ren L, Chen C, Xu H, Kim YJ, Atkinson A, Zhan Z, Sun J, Peng B, Liu L, Wang S, Cheng H, Gao J, Chen W, and Shen Y. Decoder-Hybrid-Decoder Architecture for Efficient Reasoning with Long Generation. arXiv preprint arXiv:2507.06607 2025. Introduces the SambaY architecture (Phi-4-mini-flash).

141. Yang S, Kautz J, and Hatamizadeh A. Gated Delta Networks: Improving Mamba2 with Delta Rule. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: https://arxiv.org/abs/2412.06464.